

Routing Multiple Agents Below the Sum of Distances

Anonymous author(s)

Anonymous affiliation(s)

Abstract

We study TRANSIENT MULTIAGENT PATHFINDING, a variant of the classical MULTI-AGENT PATHFINDING problem in which a set of agents must be routed without collisions from designated start vertices to designated destination vertices in a graph.

We analyze the problem within the above-and-below-guarantee paradigm of parameterized complexity. In particular, we consider the natural upper bound $\mathcal{L}$, given by the sum of the shortest-path distances between pairs of agents' terminals (corresponding to sequential routing of the agents). The parameterization is given by the gap $\zeta = \mathcal{L} - \lambda$ between this bound and the target makespan λ , together with the number k of agents. Our main result establishes fixed-parameter tractability for the combined parameter $k + \zeta$.

Matching lower bounds show that parameterization by k alone is W[1]-hard and that parameterization by ζ alone is W[1]-hard when terminals are not required to be distinct. On the positive side, if all terminals are distinct, the problem becomes fixed-parameter tractable when parameterized solely by ζ . Finally, we show that TRANSIENT MULTIAGENT PATHFINDING does not admit a polynomial kernel when parameterized by $k + \zeta$. Together, our results provide an almost complete characterization of the parameterized complexity landscape of the problem for the considered parameters.

2012 ACM Subject Classification Mathematics of computing → Combinatorial algorithms; Theory of computation → Fixed parameter tractability

Keywords and phrases Multi-Agent Pathfinding, Parameterized Complexity, Motion Planning

1 Introduction

Gnomes are not known for their sociability. They strongly prefer solitude and take great care never to encounter one another. Unfortunately for them, they must occasionally traverse a shared network of tunnels in order to reach their individual destinations.

We model the tunnel system as a graph, where vertices represent chambers and junctions and edges represent narrow passages between them. Each gnome is assigned a start chamber and a destination chamber. Time advances in discrete steps, and at each step a gnome may either move to an adjacent chamber or remain still. Any encounter between two gnomes is considered disastrous: no two gnomes may occupy the same chamber at the same time, nor may they traverse the same tunnel simultaneously, not even in opposite directions. Once a gnome reaches its destination, it immediately disappears into its private quarters and no longer interacts with the system.

A simple scenario that allows all gnomes to reach their destinations is the following. The gnomes first agree on an order and then move one after another. The first gnome traverses a shortest path to its destination; once it arrives and disappears, the second gnome starts moving, and so on. This straightforward strategy is easy to compute and yields a schedule whose makespan, that is, the total time needed to move the gnomes to their destinations, is upper bounded by 1 plus the sum of the lengths of the shortest paths of all gnomes. Achieving even a slightly smarter strategy quickly becomes nontrivial: Avoiding encounters may require gnomes to wait inside chambers, to take detours, or to carefully interleave their movements.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

The main question we address in this work is whether it is possible to obtain a schedule with a smaller makespan and how difficult this task is computationally.

Of course, while the life of a gnome is certainly an interesting subject, the main question we address arises from its connections to more practical, real-life problems.

Routing multiple agents from designated start vertices to designated destination vertices without collisions is a fundamental problem in multi-agent planning. The problem is strongly motivated by numerous real-world applications, including automated warehouse systems [29, 40], traffic management and control [33], and robotics [39]. A well-known example is the coordination of large fleets of mobile robots in Amazon fulfillment centers, where systems such as Kiva rely on efficient and reliable multi-robot path planning to transport inventory pods while avoiding collisions [40]. From a theoretical perspective, almost all variants of the problem are NP-hard [15, 35, 41].

The range of applications such as automated warehouses, aircraft towing, UAV traffic management, or drone delivery has led to the development of numerous problem variants (see, e.g., [36, 38] for surveys).

The problem is commonly formalized as MULTIAGENT PATHFINDING (MAPF), where the environment is modeled as an n -vertex graph and each of the k agents has a specified start vertex s_i and goal vertex t_i . At each discrete time step, an agent either waits or moves along an edge to a neighboring vertex; no vertex or edge may be used by more than one agent in the same time step. In the standard formulation of MAPF, the objective is to find a schedule that routes all agents to their goals while minimizing the *makespan* λ (the arrival time of the last agent) [3, 12, 13, 18, 20]. Another popular measure is the total distance traveled by the agents [10, 11, 25, 34]. Let us remark that the total distance travel by the agents is always at least the sum of the lengths of the shortest paths between all terminals.

We focus on algorithms with provable guarantees for computing schedules of optimal makespan. The problem has been extensively studied and is computationally intractable in the classical sense: it is NP-hard not only on general graphs, but also on solid grid graphs [13] and even on trees [20]. Given this intractability, numerous heuristic and algorithmic approaches have been proposed, including SAT-based methods [37], and A^* -based algorithms [28]; see also the surveys [36, 38].

There are two common assumptions regarding how agents behave upon reaching their targets; see [36]. Under the first assumption, an agent remains at its target vertex until all agents have reached their respective targets. Under the second assumption, agents disappear immediately after reaching their targets. Our results concern the latter assumption. We adopt the model in which agents enter the graph at their start times at s_i , traverse along edges, and immediately disappear upon reaching t_i ; in particular, t_i is not occupied after arrival.

In this work, we adopt the framework of parameterized complexity [17, 9]. Although the computational complexity of many fundamental coordinated motion planning variants was established several decades ago [34], recent years have witnessed renewed interest in these problems through the lens of parameterized complexity [4, 12, 14, 18, 19, 20, 21].

Most of this line of work focuses on natural parameterizations such as the number of agents, the makespan, or structural parameters of the input graph, for example its treewidth. We take a different perspective here. In particular, we follow the above-or-below-guarantee paradigm, a popular approach in parameterized complexity. The term above (below) guarantee parameterization was introduced by Mahajan and Raman [32]. This

concept has proved fruitful for studying numerous problems in parameterized complexity [1, 6, 8, 22, 24, 27, 31], see also the survey [26].

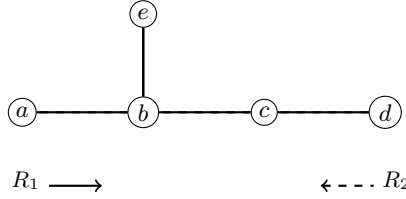
Consider the simple sequential protocol that routes the agents one after another along shortest s_i - t_i -paths. It is not difficult to show (see Lemma 3) that this protocol guarantees completion within $\lambda \leq \mathcal{L}_{\mathcal{R},G} := 1 + \sum_{i \in [k]} \text{dist}_G(s_i, t_i)$ time steps.

This bound is tight: there exist instances of the problem that require exactly $\mathcal{L}_{\mathcal{R},G}$ rounds. For example, let G be a path with end-vertices u and v , and consider two agents with terminals $s_1 = t_2 = u$ and $s_2 = t_1 = v$. In this case, neither agent can start moving before the other has reached its destination and disappeared; otherwise, they would block each other along the path. Consequently, one agent must wait until the other completes its traversal, incurring at least one additional round between their movements. Therefore, any feasible schedule has makespan at least $1 + \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) = \mathcal{L}_{\mathcal{R},G}$. Hence, the parameter $\zeta_{\mathcal{R},G} = \mathcal{L}_{\mathcal{R},G} - \lambda$ quantifies how much tighter a schedule we obtain compared to the (sometimes pessimistic) upper bound $\mathcal{L}_{\mathcal{R},G}$. In the terminology of parameterized complexity, parameterizing by $\zeta_{\mathcal{R},G}$ corresponds to *parameterization below the trivial upper bound* $\mathcal{L}_{\mathcal{R},G}$.

Before proceeding with an overview of our results, let us give an example of a possible scheduling of the agents, postponing the formal definitions to Section 2.

Example

Consider the following graph with two agents R_1 and R_2 , where R_1 needs to move from a to d and R_2 from d to a .



The upper bound $\mathcal{L}_{\mathcal{R},G}$ in this case is

$$1 + \sum_{i \in [2]} \text{dist}_G(s_i, t_i) = 1 + \text{dist}_G(a, d) + \text{dist}_G(d, a) = 1 + 3 + 3 = 7.$$

If both agents traverse the shortest path a - b - c - d starting at time 0 in opposite directions, then at some time step they traverse the same edge in opposite directions. These routes are *conflicting*. A valid (but not optimal) schedule is when first R_1 moves from a to d , and then R_2 moves from d to a . That is,

time step x	R_1	R_2
0	a	—
1	b	—
2	c	—
3	d (disappears)	—
4	—	d
5	—	c
6	—	b
7	—	a (disappears)

The makespan λ in this case is equal to 7, the moment agent R_2 reaches a . It is also equal to the upper bound $\mathcal{L}_{\mathcal{R},G}$.

XX:4 Routing Multiple Agents Below the Sum of Distances

Consider another, now optimal, scheduling, where R_1 use the side vertex e as a buffer:

time step x	R_1	R_2
0	a	d
1	b	c
2	e	b
3	b	a (disappears)
4	c	—
5	d (disappears)	—

At no time step do the agents occupy the same vertex or traverse the same edge simultaneously. Hence, the routes are non-conflicting. Agent R_1 finishes at time 5, agent R_2 at time 3, so the makespan is $\lambda = 5$, improving over the upper $\mathcal{L}_{\mathcal{R},G}$ by $\zeta_{\mathcal{R},G} = \mathcal{L}_{\mathcal{R},G} - \lambda = 2$.

Our contribution. We consider the following problem.

TRANSIENT MULTIAGENT PATHFINDING

Input: A connected graph G , integers k and λ , and a set $\mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}$ of agents.

Task: Is there a schedule for $\mathcal{R}$ of makespan at most λ ?

Our main result shows that the problem is fixed-parameter tractable with respect to the combined parameter $k + \zeta$, where k is the number of agents and ζ measures the improvement over the upper bound $\mathcal{L}$.

► **Theorem 1.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with $\lambda = \mathcal{L}_{\mathcal{R},G} - \zeta$.*

Overview of the proof of Theorem 1. The proof is based on structural properties of shortest paths. The starting point is the fact (see Lemma 7) that TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k\lambda)} \cdot n^{\mathcal{O}(1)}$ time. Thus, Theorem 1 holds if $\lambda = \mathcal{L}_{\mathcal{R},G} - \zeta = \mathcal{O}(k\zeta)$. In particular, this is fulfilled if $\max_{i \in [k]} \text{dist}_G(s_i, t_i) = \mathcal{O}(\zeta)$. We prove that solving TRANSIENT MULTIAGENT PATHFINDING can be reduced to this special case when the length of shortest s_i - t_i -paths is $\mathcal{O}(\zeta)$. For this, we consider an instance where there is $i \in [k]$ such that the distance between s_i and t_i is sufficiently large, and either demonstrate that this is a no-instance or construct a routing with makespan at most λ . Moreover, to rule out the no-instances, it is sufficient to verify whether there is $i \in [k]$ with $\text{dist}_G(s_i, t_i) > \lambda$ or there are distinct $i, j \in [k]$ such that the agents R_i and R_j cannot be routed within the makespan λ . The latter can be checked by an XP algorithm solving TRANSIENT MULTIAGENT PATHFINDING in $(kn)^{\mathcal{O}(k)}$ time (see Lemma 6). In all other cases, we are able to construct a routing schedule of the makespan at most λ .

We briefly sketch the main ideas behind constructing a routing schedule in the case when there is a pair of faraway terminals. The proof is technical and is obtained by a sequence of lemmas establishing sufficient conditions for the existence of a feasible schedule.

Given two terminal pairs (s_i, t_i) and (s_j, t_j) , we consider *laminar* shortest s_i - t_i - and s_j - t_j -paths P_i and P_j , respectively, that is, shortest paths whose intersection is either empty or a path. We distinguish *codirectional* and *opposite* paths. Paths P_i and P_j are codirectional if the agents transverse the common subpath in the same direction following P_i and P_j , and the paths are opposite, otherwise (see Figure 2). We note that the agents can follow codirectional shortest paths almost simultaneously. This implies that if there are codirectional

Parameter	Result
$k + \zeta$	FPT in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ (Theorem 1); no polynomial kernel (Theorem 17)
k	W[1]-hard (subcubic graphs) (Corollary 15); In XP (Lemma 6)
ζ	W[1]-hard (Theorem 16); Para-NP vs. XP is open

■ **Table 1** Parameterized complexity of TRANSIENT MULTIAGENT PATHFINDING.

shortest path P_i and P_j of length $\Omega(\zeta)$, then we can route the agents R_i and R_j saving considerable time, and this is sufficient for the existence of a routing of the makespan at most λ . We also observe that in some cases, opposite paths of length $\Omega(\zeta)$ can be used to save time. In particular, this happens if the common part of the paths is sufficiently far from one of the end-points of the paths.

If the above cases do not apply, we observe that if there are three terminal pairs with terminals at distance $\Omega(\zeta)$, we can find codirectional shortest paths for two pairs. Thus, the analysis boils down to the cases when the terminals of either one or two pairs are at distance at least $c \cdot \zeta$ for a specific constant $c > 0$, and for all other pairs (s_i, t_i) , the distance between s_i and t_i is upper bounded by $c' \cdot \zeta$ for another constant $c' < c$. If there is exactly one shortest path of length at least $c \cdot \zeta$, then we can route the corresponding agent R_i along a shortest path. Simultaneously with R_i , we can route other agents and save time.

The case of two pairs of faraway terminals is the most challenging. Here, we need some additional structural results about paths in G . Very roughly, the idea is to route two agents R_i and R_j with $\text{dist}_G(s_i, t_i), \text{dist}_G(s_j, t_j) \geq c \cdot \zeta$ in the optimum way (which is proved to be very special after excluding some cases), and route the other agents while R_i and R_j follow their routes. This concludes the proof overview.

Of course, a natural question is whether Theorem 1 is tight. In other words, whether the combined parameterization is necessary. We complement our theorem with lower bounds showing that this is indeed the case. First, we observe (Theorem 14 and Corollary 15) that TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard when parameterized by k , even if the input graph is subcubic. We also prove (Theorem 16) that TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard when parameterized by ζ . Therefore, fixed-parameter tractability with respect to only one of these parameters is highly unlikely. Regarding single-parameter parameterizations, we observe (Lemma 6) that for fixed k , TRANSIENT MULTIAGENT PATHFINDING is solvable in polynomial time $(kn)^{\mathcal{O}(k)}$. That is, it belongs to XP when parameterized by k . Whether the problem is Para-NP-complete or in XP when parameterized by ζ remains an interesting open question.

Having established an FPT result, a natural next question concerns the existence of a polynomial kernel. We show in Theorem 17 that TRANSIENT MULTIAGENT PATHFINDING parameterized by $k + \zeta$ does not admit a polynomial kernel. Combined with the above lower bounds, Theorem 1 yields an almost complete characterization of the parameterized complexity of TRANSIENT MULTIAGENT PATHFINDING. We summarize these results in Table 1.

Although our lower bounds exclude the existence of an FPT algorithm for TRANSIENT MULTIAGENT PATHFINDING parameterized solely by k , we identify a natural subclass that does admit such algorithms. In particular, when all terminals are distinct, the upper bound on λ improves to $\mathcal{L}_{\mathcal{R},G}^* = \sum_{i \in [k]} \text{dist}(s_i, t_i)$. We establish the following theorem.

► **Theorem 2.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(\zeta^3)} \cdot n^{\mathcal{O}(1)}$*

time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with pairwise distinct terminals and $\lambda = \mathcal{L}_{\mathcal{R}, G}^* - \zeta$.

2 Preliminaries

We use $\mathbb{N}$ to denote the set of non-negative integers, and write $[p]$ and $[0, p]$ for the set $\{1, \dots, p\}$ and $\{0, \dots, p\}$, respectively, where $p \in \mathbb{N}$. We refer to the textbook [16] for the basic graph-theoretic notions and notation. We are given an undirected graph G and a set $\mathcal{R} = \{R_1, R_2, \dots, R_k\}$ of k agents. Each $R_i \in \mathcal{R}$ has a starting vertex s_i and a destination vertex t_i in $V(G)$ such that $s_i \neq t_i$. We refer to the elements in the set $\{s_i \mid i \in [k]\} \cup \{t_i \mid i \in [k]\}$ as *terminals*. We use a discrete time frame $[0, t]$, $t \in \mathbb{N}$, to reference the sequence of moves of the agents, and in each time step $x \in [0, t]$, every agent $R_i \in \mathcal{R}$ performs one of the following actions:

- R_i enters the graph at s_i ;
- R_i moves from its current position $v \in V(G)$ to a neighbor of v ;
- R_i stays at its current position $v \in V(G)$; or
- R_i disappears from the graph if its current position is t_i .

A route for agent R_i is a pair (W_i, x_i) , where W_i is a sequence $(u_0, \dots, u_{\delta_i})$ of vertices in G and such that

- u_0 is the starting vertex s_i of R_i ;
 - u_{δ_i} is the destination vertex t_i of R_i ;
 - for each $y \in [\delta_i]$ it holds that $u_{y-1} = u_y$ or $u_{y-1}u_y$ is an edge in G ,
- and $x_i \in \mathbb{N}$. The vertex u_y is the position of R_i at time step $x_i + y$. Thus, R_i appears at moment x_i in s_i and moves along W_i from s_i to t_i ; the agent disappears when it reaches t_i . A vertex $v \in V(G)$ is *free* at time step $x \in [0, t]$ if no agent is located at v at time step x ; otherwise, v is *occupied*.

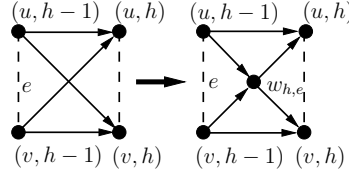
For $1 \leq i < j \leq k$, two routes $(W_i = (u_0, \dots, u_{\delta_i}), x_i)$ and $(W_j = (v_0, \dots, v_{\delta_j}), x_j)$ are *conflicting* (or *in conflict*) if there exists some time step x such that R_i and R_j are either at the same vertex or traversing the same edge at the time step x . More formally, $x = x_i + y = x_j + z$ such that either $u_y = v_z$ or $u_{y-1}u_y = v_{z-1}v_z$. A *schedule* Γ for $\mathcal{R}$ is a set of pairwise non-conflicting routes $\{(W_i, x_i) \mid i \in [k]\}$, during a time interval $[0, \lambda]$ (that is, $\max_{i \in [k]}(x_i + \delta_i) = \lambda$). The integer λ is called the *makespan* of Γ . Using the introduced terminology, we formalize TRANSIENT MULTIAGENT PATHFINDING stated in Section 1.

For the introduction to Parameterized Complexity, we refer to the textbooks [9, 17].

3 Basic observations

In this section, we collect basic observations on TRANSIENT MULTIAGENT PATHFINDING. First, we formally prove our upper bound for the minimum makespan and show some useful properties of schedules guaranteeing such a makespan.

- **Lemma 3.** *Let $(G, k, \mathcal{R}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING. If $\lambda \geq 1 + \sum_{i \in [k]} \text{dist}(s_i, t_i)$, then $(G, k, \mathcal{R}, \lambda)$ is a yes-instance. Moreover, there is a schedule of makespan at most λ such that for every $i \in [k]$,*
- (i) *the route of R_i is (P_i, x_i) where P_i is an arbitrary shortest s_i - t_i -path, and*
 - (ii) *for each internal vertex v of P_i and the moment at which R_i is in v , R_i is an only agent on G .*



291 **Figure 1** Arc modifications in the construction of $\mathcal{G}_\lambda^*$.

253 **Proof.** We assume that the agents $R_1, \dots, R_k$ are ordered in such a way that $s_i \neq t_{i-1}$ holds
 254 for all $i \in [2, \ell]$, with $\ell \leq k$ chosen as large possible. We navigate the agents $R_1, \dots, R_\ell$
 255 sequentially on shortest paths, such that we start the route of the first agent at the moment
 256 0 and the route of agent $i \geq 2$ exactly at the same time when the $(i-1)$ -th agent reaches
 257 its destination (that is at the time step $\sum_{j \in [i-1]} \text{dist}_G(s_j, t_j)$). It is straightforward that the
 258 routes in this (partial) schedule have no conflicts. If $\ell = k$ then we have the schedule with
 259 the makespan $\sum_{i \in [k]} \text{dist}_G(s_i, t_i)$. Assume that this is not the case and $\ell < k$.

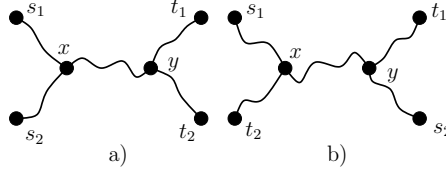
260 By the choice of ℓ , we have that for every $i \in [\ell+1, k]$, $s_i = t_\ell$ as, otherwise, we would
 261 be able to extend the ordering $R_1, \dots, R_\ell$ by appending one of the remaining agents. We
 262 also have that for every $i \in [\ell+1, k]$, $t_i = s_1$ because we would be able to insert the agent
 263 with $t_i \neq s_1$ for $i \in [\ell+1, k]$ in the beginning of the ordering, contradicting the choice of ℓ .
 264 Note that this implies that $s_1 \neq t_\ell$ as $s_i \neq t_i$ for all $i \in [k]$. Then, we can extend the partial
 265 schedule already constructed for $R_1, \dots, R_\ell$ as follows. We set $t = \sum_{i \in [\ell]} \text{dist}_G(s_i, t_i) + 1$,
 266 and then we navigate the agents $R_\ell, \dots, R_k$ using the same strategy as for the previous
 267 agent starting from the moment t . More precisely, for each $i \in [\ell+1, k]$ we move R_i from s_i
 268 to t_i along the shortest path, and R_i starts moving at the moment $1 + \sum_{j \in [i-1]} \text{dist}_G(s_j, t_j)$.
 269 Because $s_i = t_\ell$ and $t_i = s_1 \neq s_i$ for each $i \in [\ell+1, k]$ and R_ℓ starts to move at the
 270 moment $t + 1$, the constructed schedule is feasible, that is, the routes have no conflicts.
 271 Because the makespan is $1 + \sum_{i \in [k]} \text{dist}_G(s_i, t_i)$, we have that $(G, k, \mathcal{R}, \lambda)$ is a yes-instance
 272 if $\lambda \geq 1 + \sum_{i \in [k]} \text{dist}(s_i, t_i)$. It is straightforward to see that the constructed schedule satisfies
 273 requirements (i) and (ii). This concludes the proof. $\blacktriangleleft$

274 In Theorem 1, we exploit the facts that TRANSIENT MULTIAGENT PATHFINDING is in XP
 275 when parameterized by k and in FPT for the parameterization by k and λ . To show these
 276 properties, we reduce the problem to finding paths in auxiliary time-expansion graphs.

277 Let G be a graph and let $\lambda \geq 0$ be an integer. We denote by $\mathcal{G}_\lambda$ the directed *time-*
 278 *expansion* graph with the vertex set $\{(v, t) \mid v \in V(G) \text{ and } t \in [0, \lambda]\}$ where there is an
 279 arc from a vertex (u, i) to a vertex (v, j) if and only if $j - i = 1$ and $v \in N_G[u]$. In other
 280 words, the set of vertices consists of $\lambda + 1$ copies of $V(G)$ forming layers, arcs join vertices of
 281 subsequent layers, and (u, i) from the i -th layer is joined with arcs with the copies of the
 282 vertices v of G in the $(i+1)$ -th layer that are either the same as u or adjacent to u in G .
 283 Notice that $\mathcal{G}_\lambda$ is a directed acyclic graph (DAG). The construction of $\mathcal{G}_\lambda$ and the definition
 284 of the routes in TRANSIENT MULTIAGENT PATHFINDING imply the following observation.

285 **► Observation 4.** An instance $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ of TRANSIENT MUL-
 286 TIAGENT PATHFINDING is a yes-instance if and only if there are integers $p_1, \dots, p_k \in [0, \lambda]$
 287 and $q_1, \dots, q_k \in [0, \lambda]$ such that $\mathcal{G}_\lambda$ has vertex disjoint directed paths $P_1, \dots, P_k$, where P_i is
 288 an (s_i, p_i) - (t_i, q_i) -path for every $i \in [k]$, with the additional property that for every $h \in [\lambda]$
 289 and each edge $\{u, v\}$ of G , the arcs $((u, h-1), (v, h))$ and $((v, h-1), (u, h))$ are not used
 290 by two paths.

XX:8 Routing Multiple Agents Below the Sum of Distances



322 ■ **Figure 2** Laminar codirectional (a) and opposite (b) s_1 - t_1 and s_2 - t_2 -paths.

292 Furthermore, we can augment $\mathcal{G}_\lambda$ by artificial terminals and modify the graph to encode
 293 the additional property to reduce TRANSIENT MULTIAGENT PATHFINDING to the DISJOINT
 294 PATHS problem. We remind the reader that the task of DISJOINT PATHS is, given a (directed)
 295 graph G and k pairs of terminal vertices (s_i, t_i) for $i \in [k]$, to decide whether there are k
 296 vertex disjoint paths joining these pairs of terminals. Let $s_i, t_i \in V(G)$ be terminal vertices
 297 for $i \in [k]$. We construct $\mathcal{G}_\lambda^*$ as follows:

- 298 ■ construct $\mathcal{G}_\lambda$ and $2k$ vertices s_i^* and t_i^* for $i \in [k]$,
- 299 ■ for each $i \in [k]$ and every $j \in [0, \lambda]$, construct arcs $(s_i^*, (s_i, j))$ and $((t_i, j), t_i^*)$,
- 300 ■ for each $h \in [\lambda]$ and every edge $e = \{u, v\}$ of G , construct a vertex $w_{h,e}$ and replace the arcs
 301 $((u, h-1), (v, h))$ and $((v, h-1), (u, h))$ by four arcs $((u, h-1), w_{h,e})$, and $(w_{h,e}, (v, h))$,
 302 $((v, h-1), w_{h,e})$, $(w_{h,e}, (u, h))$;

303 see Figure 1 for the arc modifications in the construction. Notice that $\mathcal{G}_\lambda^*$ is a DAG. By
 304 Observation 4, we obtain the following claim. Due to space constraints, some proof details
 305 are deferred to the full version. Affected results are marked with a $(\star)$.

306 ► **Observation 5** $(\star)$. *An instance $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ of TRANSIENT*
 307 *MULTIAGENT PATHFINDING is a yes-instance if and only if $\mathcal{G}_\lambda^*$ has vertex disjoint directed*
 308 *paths $P_1, \dots, P_k$, where P_i is an s_i^* - t_i^* -path for every $i \in [k]$.*

309 Because DISJOINT PATHS can be solved in $n^{\mathcal{O}(k)}$ time for k pairs of terminals on DAGs
 310 by the result of Fortune, Hopcroft, and Wyllie [23], by Observation 5, we have the following.

311 ► **Lemma 6** $(\star)$. *TRANSIENT MULTIAGENT PATHFINDING is solvable in time $(kn)^{\mathcal{O}(k)}$.*

312 Observation 5 also allows us to show that TRANSIENT MULTIAGENT PATHFINDING is
 313 FPT when parameterized by k and λ by making use of the *color coding* technique of Alon,
 314 Yuster, and Zwick [2].

315 ► **Lemma 7** $(\star)$. *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k\lambda)} \cdot n^{\mathcal{O}(1)}$*
 316 *time.*

317 4 Transient Multiagent Pathfinding parameterized below $\mathcal{L}$

318 In this section, we prove our main result, Theorem 1: TRANSIENT MULTIAGENT PATHFINDING
 319 is FPT when parameterized by k and $\zeta = \mathcal{L}_{\mathcal{R},G} - \lambda$. Throughout the section, we assume that
 320 $\lambda \leq \mathcal{L}_{\mathcal{R},G}$ in the considered instances (otherwise, we have a trivial yes-instance by Lemma 3).
 321 We start with some notation and auxiliary results.

322 Let (s_1, t_1) and (s_2, t_2) be pairs of distinct vertices of a graph G . We say that an
 323 s_1 - t_1 -path P_1 and s_2 - t_2 -path P_2 are *laminar* if either they have no common vertices or
 324 the intersection of P_1 and P_2 is a path in G (see Figure 2). Laminar paths P_1 and P_2
 325 are *codirectional* if either they are disjoint or their intersection is an x - y -path Q where
 326 either $\text{dist}_G(s_1, x) \leq \text{dist}_G(s_1, y)$ and $\text{dist}_G(s_2, x) \leq \text{dist}_G(s_2, y)$ or $\text{dist}_G(s_1, y) \leq \text{dist}_G(s_1, x)$
 327 either $\text{dist}_G(s_1, x) \leq \text{dist}_G(s_1, y)$ and $\text{dist}_G(s_2, x) \leq \text{dist}_G(s_2, y)$ or $\text{dist}_G(s_1, y) \leq \text{dist}_G(s_1, x)$

and $\text{dist}_G(s_2, y) \leq \text{dist}_G(s_2, x)$ (see Figure 2(a)). Otherwise, we say that P_1 and P_2 are opposite (see Figure 2(b)). We make the following easy observation.

► **Observation 8** ($\star$). *For any two pairs (s_1, t_1) and (s_2, t_2) of distinct vertices of a graph G , G has laminar shortest s_1 - t_1 - and s_2 - t_2 -paths. Moreover, for any given shortest s_1 - t_1 and s_2 - t_2 -paths P_1 and P_2 , there is a shortest s_1 - t_1 -path P'_1 such that P'_1 and P_2 are laminar.*

In the series of the next lemmas, we give sufficient conditions ensuring that an instance of TRANSIENT MULTIAGENT PATHFINDING is a yes-instance.

We first show that the existence of three pairs of terminals at distances $\Omega(\zeta)$ is a sufficient condition for having a yes-instance.

► **Lemma 9** ($\star$). *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Suppose that there are distinct $h, i, j \in [k]$, such that*

- (i) $\text{dist}_G(s_h, t_h) \geq 2\zeta + 3$,
 - (ii) $\text{dist}_G(s_i, t_i) \geq \zeta + 2$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 2$.
- Then, $\mathcal{I}$ is a yes-instance.

Next, we give sufficient conditions for a yes-instance when there is a pair of terminals at sufficiently big distance.

► **Lemma 10** ($\star$). *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Let also $I = \{i \in [k] \mid \text{dist}_G(s_i, t_i) \leq \zeta + 1\}$. Suppose that*

- (i) *there is an $h \in [k]$ such that $5\zeta + 4 \leq \text{dist}_G(s_h, t_h) \leq \mathcal{L}_{\mathcal{R}, G} - \zeta$,*
 - (ii) $\sum_{i \in I} \text{dist}_G(s_i, t_i) \geq \zeta$.
- Then, $\mathcal{I}$ is a yes-instance. Furthermore, if
- (a) *there is an $h \in [k]$ such that $3\zeta + 2 \leq \text{dist}_G(s_h, t_h)$,*
 - (b) *either (i) $|[k] \setminus I| = 1$ and $\text{dist}_G(s_h, t_h) \leq \mathcal{L}_{\mathcal{R}, G} - \zeta$ or (ii) $|[k] \setminus I| > 1$ and $1 + \sum_{i \in [k] \setminus I} \text{dist}_G(s_i, t_i) \leq \mathcal{L}_{\mathcal{R}, G} - \zeta$, and*
 - (c) $\sum_{i \in I} \text{dist}_G(s_i, t_i) < \zeta$,
- then $\mathcal{I}$ is a yes-instance.

In the following lemma, we provide sufficient conditions for a yes-instance when there are exactly two pairs of terminals at sufficiently big distances.

► **Lemma 11** ($\star$). *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Suppose that there are distinct $i, j \in [k]$ such that*

- (i) $\text{dist}_G(s_i, t_i) \geq 5\zeta + 5$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 2$,
 - (ii) *the agents R_i and R_j can be routed within the makespan $\mathcal{L}_{\mathcal{R}, G} - \zeta$ without conflicts with each other,*
 - (iii) *for every $h \in [k]$ such that $h \neq i, j$, $\text{dist}_G(s_h, t_h) \leq \zeta + 1$.*
- Then, $\mathcal{I}$ is a yes-instance.

Now we are ready to prove Theorem 1. We restate it here.

► **Theorem 1.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with $\lambda = \mathcal{L}_{\mathcal{R}, G} - \zeta$.*

XX:10 Routing Multiple Agents Below the Sum of Distances

Proof. Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. If $\zeta = 0$, then $\mathcal{I}$ is a yes-instance by Lemma 3. Assume that $\zeta \geq 1$. We also assume that $\text{dist}_G(s_1, t_1) \geq \dots \geq \text{dist}_G(s_k, t_k)$.

If $\text{dist}_G(s_1, t_1) > \lambda$, then we have a trivial no-instance. If $k \geq 2$, we use the algorithm from Lemma 6 to check whether $(G, 2, \{R_1, R_2\}, \lambda)$ is a yes-instance. If $k = 2$, then we solved the problem. If $k \geq 3$ and we got the no-answer, then $\mathcal{I}$ is a no-instance. Note that the algorithm runs in polynomial time for two agents. From now on, we assume that $k \geq 3$ and there is a schedule for R_1 and R_2 with makespan at most λ .

Next, we consider the cases depending on the distances between terminals and either use Lemma 9, Lemma 10, Lemma 11 to argue that $\mathcal{I}$ is a yes-instance or apply Lemma 7 to solve the problem.

Suppose first that $\text{dist}_G(s_1, t_1) \leq 2\zeta + 2$. Then, we have $\sum_{i=1}^k \text{dist}_G(s_i, t_i) \leq (2\zeta + 2)k$ and $\lambda \leq (2\zeta + 2)k$ as $\zeta \geq 1$. In this case, we apply the algorithm from Lemma 7 and solve the problem in $2^{k^2\zeta} \cdot n^{\mathcal{O}(1)}$ time. If $\text{dist}_G(s_1, t_1) \geq 2\zeta + 3$ and $\text{dist}_G(s_2, t_2) \geq \zeta + 2$, then we conclude that $\mathcal{I}$ is a yes-instance by Lemma 9. Thus, from now on, we assume that $\text{dist}_G(s_i, t_i) \leq \zeta + 1$ for each $i \in [3, k]$.

Suppose that $\text{dist}_G(s_1, t_1) \leq 5\zeta + 3$. Then,

$$\lambda \leq \sum_{i=1}^k \text{dist}_G(s_i, t_i) \leq 2(5\zeta + 3) + (k-2)(\zeta + 1) = k(\zeta + 1) + 8\zeta + 4.$$

We apply the algorithm from Lemma 7 and solve the problem in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ time.

From now on, assume that $\text{dist}_G(s_1, t_1) \geq 5\zeta + 4$. If $\text{dist}_G(s_2, t_2) \leq \zeta + 1$, then $\mathcal{I}$ is a yes-instance by Lemma 10: if $\sum_{i=2}^k \text{dist}_G(s_i, t_i) \geq \zeta$ then we apply the first claim and use the second claim otherwise. Furthermore, if $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \geq \zeta$ then $\mathcal{I}$ is a yes-instance even if $\text{dist}_G(s_2, t_2) \geq \zeta + 2$ by the same lemma.

Now, we may assume $\text{dist}_G(s_2, t_2) \geq \zeta + 2$ and $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \leq \zeta - 1$. If $\text{dist}_G(s_1, t_1) \geq 5\zeta + 5$, then $\mathcal{I}$ is a yes-instance by Lemma 11. Hence, it remains to consider the case when $\text{dist}_G(s_1, t_1) \leq 5\zeta + 4$. Here, we have that $\lambda \leq \sum_{i=1}^k \text{dist}_G(s_i, t_i) \leq 2(5\zeta + 4) + \zeta - 1 = 11\zeta + 7$ and the problem can be solved in $2^{\mathcal{O}(k\zeta)} \cdot n^{\mathcal{O}(1)}$ time by Lemma 7. This concludes the case analysis.

Since the distances between terminals can be computed in polynomial time by standard algorithms (see, e.g., [7]), the overall running time is $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$, concluding the proof. ◀

5 The case of distinct terminals

In this section, we prove that TRANSIENT MULTIAGENT PATHFINDING is FPT when parameterized by only $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$ if the terminals are pairwise distinct. The result is based on the following lemma.

► **Lemma 12** (★). *Let G be a connected graph and let $R_1 = (s_1, t_1)$, $R_2 = (s_2, t_2)$, and $R_3 = (s_3, t_3)$ be agents such that the terminal vertices $s_1, s_2, s_3, t_1, t_2, t_3$ are distinct. Then there are distinct $i, j \in [3]$ such that the agents R_i and R_j can be routed with the makespan at most $\text{dist}_G(s_i, t_i) + \text{dist}_G(s_j, t_j) - 1$ without conflicts.*

By Lemma 12, we can show the following.

► **Lemma 13.** *Let $(G, k, \mathcal{R}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING such that the terminals are pairwise distinct. If $\lambda \geq \sum_{i \in [k]} \text{dist}(s_i, t_i) - \lfloor \frac{k-1}{2} \rfloor$, then $(G, k, \mathcal{R}, \lambda)$ is a yes-instance.*

Proof. If $k = 1$ then the claim is trivial. If $k = 2$ then we route R_1 and R_2 using shortest s_1 - t_1 - and s_2 - t_2 -paths P_1 and P_2 , respectively, using the routes $(P_1, 0)$, $(P_2, \text{dist}_G(s_1, t_1))$. Since $t_1 \neq s_2$, there are no conflicts and this proves the claim. Assume that $k \geq 3$.

We greedily select $r = \lfloor \frac{k-1}{2} \rfloor$ pairs $\{i_1, j_1\}, \dots, \{i_r, j_r\}$ of indices such that all the indices are distinct and for every $h \in [r]$, the agents R_{i_h} and R_{j_h} can be routed with the makespan at most $\text{dist}_G(s_{i_h}, t_{i_h}) + \text{dist}_G(s_{j_h}, t_{j_h}) - 1$ without conflicts. This is possible due to Lemma 12. We fix these routes and concatenate them, that is R_{i_1} and R_{j_1} start at the moment 0, and for each $h \in [2, r]$, R_{i_h} and R_{j_h} are routed when $R_{i_{h-1}}$ and $R_{j_{h-1}}$ reach their destinations. Then, we consecutively append the routes for $h \in [k] \setminus (\{i_1, \dots, i_r\} \cup \{j_1, \dots, j_r\})$ using shortest s_h - t_h -paths. Because the terminals are distinct, we obtain a feasible schedule with the total makespan at most $\sum_{i \in [k]} \text{dist}(s_i, t_i) - \lfloor \frac{k-1}{2} \rfloor$. This proves the lemma. ◀

We remark that Lemma 13 implies that if the terminals are distinct then $(G, k, \mathcal{R}, \lambda)$ is a yes-instance of TRANSIENT MULTIAGENT PATHFINDING if $\lambda \leq \mathcal{L}_{\mathcal{R}, G}^* := \sum_{i \in [k]} \text{dist}(s_i, t_i)$.

Combining Theorem 1 and Lemma 13, we obtain the algorithmic result about TRANSIENT MULTIAGENT PATHFINDING parameterized below $\mathcal{L}_{\mathcal{R}, G}^*$ for distinct terminals.

► **Theorem 2.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(\zeta^3)} \cdot n^{\mathcal{O}(1)}$ time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with pairwise distinct terminals and $\lambda = \mathcal{L}_{\mathcal{R}, G}^* - \zeta$.*

Proof. Let $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING with pairwise distinct terminals and $\lambda = \mathcal{L}_{\mathcal{R}, G}^* - \zeta$. If $\zeta \leq \lfloor \frac{k-1}{2} \rfloor$, this is a yes-instance by Lemma 13. Otherwise, $k \leq 2\zeta$ and we can apply the algorithm from Theorem 1 and solve the problem in $2^{\mathcal{O}(\zeta^3)} \cdot n^{\mathcal{O}(1)}$ time. ◀

6 Computational lower bounds

In this section, we prove our hardness results. For this, it will be convenient to consider LAYERED VERTEX-DISJOINT SHORTEST PATHS [5]. A graph is layered if there is a partition of the vertices into (disjoint) sets $V_1, V_2, \dots, V_\lambda$ such that all edges $\{u, v\}$ are between two consecutive layers, that is $u \in V_i$ and $v \in V_{i+1}$ or $u \in V_{i+1}$ and $v \in V_i$ for some $i \in [\lambda - 1]$. An instance of LAYERED VERTEX-DISJOINT SHORTEST PATHS consists of a layered graph and k terminal pairs (s_i, t_i) such that $s_i \in V_1$, $t_i \in V_\lambda$, and each shortest s_i - t_i -path contains exactly one vertex of each layer. The question is whether there are k shortest paths (one between each terminal pair) that are pairwise vertex-disjoint. It is known that LAYERED VERTEX-DISJOINT SHORTEST PATHS does not admit a polynomial kernel parameterized by $k + \lambda$ (the number of terminal pairs plus the number of layers) [5]. Moreover, parameterized by only k , the problem is known to be W[1]-hard [30]. We present a reduction from LAYERED VERTEX-DISJOINT SHORTEST PATHS to TRANSIENT MULTIAGENT PATHFINDING based on a reduction by [20, 21].

► **Theorem 14.** *TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard parameterized by k .*

Proof. We reduce from LAYERED VERTEX-DISJOINT SHORTEST PATHS parameterized by the number k' of terminal pairs, which is known to be W[1]-hard [30]. Let λ' be the number of layers in the input graph. For each terminal pair (s_i, t_i) , we construct an agent $R_i = (s_i, t_i)$ and we set $\lambda = \lambda' - 1$. We next show that the reduction is correct. First assume that there is a set of k vertex-disjoint shortest paths. Then, we let each agent appear in time step 0, and let them move along the shortest path until reaching t_i in time step $\lambda' - 1 = \lambda$. Since all paths are vertex-disjoint, this shows that the constructed instance is a yes-instance.

XX:12 Routing Multiple Agents Below the Sum of Distances

455 In the other direction, assume that there is a schedule for all agents. Since the distance
456 between s_i and t_i for each agent is by assumption $\lambda' - 1 = \lambda$, the schedule has to let all
457 agents appear at their respective starting vertices at time step 0 and let them move along
458 their respective shortest paths towards their destinations. Hence, each agent is in layer i
459 at time step $i - 1$ in the assumed schedule. This shows that the paths taken by all agents
460 are pairwise vertex-disjoint and thus the original instance of LAYERED VERTEX-DISJOINT
461 SHORTEST PATHS is a yes-instance. ◀

462 We mention in passing that the above result can also be modified to show $W[1]$ -hardness
463 with respect to k even if the input graph is subcubic as shown by [21]. Simply replace each
464 vertex by an in-tree followed by an out-tree (both of height $\log(n)$) and replacing each edge
465 by an edge between a unique out-leaf of the vertex in the earlier layer and a unique in-leaf of
466 the vertex in the latter layer.

467 ► **Corollary 15.** *TRANSIENT MULTIAGENT PATHFINDING is $W[1]$ -hard when parameterized*
468 *by k even if the input graph is subcubic.*

469 We next show $W[1]$ -hardness parameterized by ζ , the difference between the upper bound
470 on the makespan and λ .

471 ► **Theorem 16 (★).** *TRANSIENT MULTIAGENT PATHFINDING is $W[1]$ -hard parameterized*
472 *by ζ .*

473 We next prove Theorem 17, that it is unlikely that TRANSIENT MULTIAGENT PATHFIND-
474 ING has a polynomial kernel parameterized by $k + \zeta$. This result is shown combining ideas
475 behind Theorems 14 and 16.

476 ► **Theorem 17 (★).** *TRANSIENT MULTIAGENT PATHFINDING parameterized by $k + \zeta$ does*
477 *not admit a polynomial kernel unless $\text{coNP} \subseteq \text{NP}/\text{poly}$.*

478 7 Conclusion and open questions

479 In this work, we investigated TRANSIENT MULTIAGENT PATHFINDING through the lens of
480 the above-and-below-guarantee paradigm of parameterized complexity. Our results provide
481 a near-complete understanding of the problem with respect to the parameter k and ζ . We
482 established fixed-parameter tractability for the combined parameterization and complement
483 this with matching lower bounds showing that neither k nor ζ alone is likely to yield an FPT
484 algorithm. In addition, we ruled out the existence of a polynomial kernel for the combined
485 parameter. Together, these results delineate the boundary between tractability and hardness
486 for the problem in a rather precise manner.

487 We conclude with several open questions that we find particularly interesting.

488 ■ **Alternative upper bounds.** From a combinatorial perspective, we were unable to
489 prove or disprove whether the following inequality

$$490 \quad \lambda \leq 2 \cdot \max_{i \in [k]} \text{dist}(s_i, t_i) + k$$

491 holds for the minimum makespan λ in general. Establishing such a bound (or providing
492 a counterexample) would be of independent interest. A positive answer would, of course,
493 give rise to new algorithmic questions within the above-and-below-guarantee framework.

- **Parameterization by ζ .** The precise complexity of TRANSIENT MULTIAGENT PATHFINDING when parameterized solely by ζ remains unresolved. Is the problem in XP or para-NP-hard? Clarifying this would complete the single-parameter landscape.
- **Restricted graph classes.** Another promising direction is to study the complexity of the problem on restricted graph classes, such as planar graphs or graphs of bounded degree. In particular, it remains open whether the problem is FPT when parameterized by k on planar graphs or even on planar grids.

References

- 1 Noga Alon, Gregory Gutin, Eun Jung Kim, Stefan Szeider, and Anders Yeo. Solving MAX- r -SAT above a tight lower bound. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 511–517. SIAM, 2010.
- 2 Noga Alon, Raphael Yuster, and Uri Zwick. Color-coding. *J. ACM*, 42(4):844–856, 1995. URL: <http://dx.doi.org/10.1145/210332.210337>, doi:10.1145/210332.210337.
- 3 Jacopo Banfi, Nicola Basilico, and Francesco Amigoni. Intractability of time-optimal multirobot path planning on 2D grid graphs with holes. *IEEE Robotics and Automation Letters*, 2(4):1941–1947, 2017.
- 4 Bahareh Banyassady, Mark de Berg, Karl Bringmann, Kevin Buchin, Henning Fernau, Dan Halperin, Irina Kostitsyna, Yoshio Okamoto, and Stijn Slot. Unlabeled multi-robot motion planning with tighter separation bounds. In *SoCG*, volume 224 of *LIPIcs*, pages 12:1–12:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
- 5 Matthias Bentert, Fedor V. Fomin, and Petr A. Golovach. Tight approximation and kernelization bounds for vertex-disjoint shortest paths. In *Proceedings of the 42nd International Symposium on Theoretical Aspects of Computer Science (STACS)*, pages 17:1–17:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPICs.STACS.2025.17.
- 6 Ivona Bezáková, Radu Curticapean, Holger Dell, and Fedor V. Fomin. Finding detours is fixed-parameter tractable. *SIAM J. Discrete Math.*, 33(4):2326–2345, 2019. doi:10.1137/17M1148566.
- 7 Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009. URL: <http://mitpress.mit.edu/books/introduction-algorithms>.
- 8 Robert Crowston, Mark Jones, Gabriele Muciaccia, Geevarghese Philip, Ashutosh Rai, and Saket Saurabh. Polynomial kernels for lambda-extendible properties parameterized above the Poljak-Turzik bound. In *IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, volume 24 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 43–54, Dagstuhl, Germany, 2013. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- 9 Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michal Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015. doi:10.1007/978-3-319-21275-3.
- 10 Argyrios Deligkas, Eduard Eiben, Robert Ganian, Iyad Kanj, Dominik Leko, and M. S. Ramanujan. Routing few robots in a crowded network. *J. Comput. Syst. Sci.*, 157:103753, 2026. URL: <https://doi.org/10.1016/j.jcss.2025.103753>, doi:10.1016/J.JCSS.2025.103753.
- 11 Argyrios Deligkas, Eduard Eiben, Robert Ganian, Iyad Kanj, and M. S. Ramanujan. Parameterized algorithms for coordinated motion planning: Minimizing energy. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on Automata, Languages, and Programming, ICALP 2024, Tallinn, Estonia, July 8-12, 2024*, volume 297 of *LIPIcs*, pages 53:1–53:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024. Full version available at <https://arxiv.org/abs/2404.15950>.

XX:14 Routing Multiple Agents Below the Sum of Distances

- 12 Argyrios Deligkas, Eduard Eiben, Robert Ganian, Iyad Kanj, and M. S. Ramanujan. Parameterized algorithms for multiagent pathfinding on trees. In Sanmay Das, Ann Nowé, and Yevgeniy Vorobeychik, editors, *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2025, Detroit, MI, USA, May 19-23, 2025*, pages 584–592. International Foundation for Autonomous Agents and Multiagent Systems / ACM, 2025. URL: <https://dl.acm.org/doi/10.5555/3709347.3743574>, doi:10.5555/3709347.3743574.
- 13 Erik D. Demaine, Sándor P. Fekete, Phillip Keldenich, Henk Meijer, and Christian Scheffer. Coordinated motion planning: Reconfiguring a swarm of labeled robots with bounded stretch. *SIAM Journal on Computing*, 48(6):1727–1762, 2019.
- 14 Erik D. Demaine, Mohammad Taghi Hajiaghayi, and Dániel Marx. Minimizing movement: Fixed-parameter tractability. *ACM Trans. Algorithms*, 11(2):14:1–14:29, 2014. doi:10.1145/2650247.
- 15 Erik D. Demaine and Mikhail Rudoy. A simple proof that the $(n^2 - 1)$ -puzzle is hard. *Theoretical Computer Science*, 732:80–84, 2018.
- 16 Reinhard Diestel. *Graph theory*, volume 173 of *Graduate Texts in Mathematics*. Springer-Verlag, Berlin, 5th edition, 2017.
- 17 Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*. Texts in Computer Science. Springer, 2013.
- 18 Eduard Eiben, Robert Ganian, and Iyad Kanj. The parameterized complexity of coordinated motion planning. In Erin W. Chambers and Joachim Gudmundsson, editors, *39th International Symposium on Computational Geometry, SoCG 2023, June 12-15, 2023, Dallas, Texas, USA*, volume 258 of *LIPIcs*, pages 28:1–28:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. URL: <https://doi.org/10.4230/LIPIcs.SocG.2023.28>, doi:10.4230/LIPIcs.SocG.2023.28.
- 19 Eduard Eiben, Robert Ganian, Iyad Kanj, and M. S. Ramanujan. A minor-testing approach for coordinated motion planning with sliding robots. In Oswin Aichholzer and Haitao Wang, editors, *41st International Symposium on Computational Geometry, SoCG 2025, Kanazawa, Japan, June 23-27, 2025*, volume 332 of *LIPIcs*, pages 44:1–44:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. URL: <https://doi.org/10.4230/LIPIcs.SocG.2025.44>, doi:10.4230/LIPIcs.SocG.2025.44.
- 20 Foivos Fioravantes, Dušan Knop, Jan Matyáš Křišťan, Nikolaos Melissinos, and Michal Opler. Exact algorithms and lowerbounds for multiagent path finding: Power of treelike topology. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence*, pages 17380–17388. AAAI Press, 2024. URL: <https://doi.org/10.1609/aaai.v38i16.29686>.
- 21 Foivos Fioravantes, Dusan Knop, Jan Matyáš Kristan, Nikolaos Melissinos, and Michal Opler. Exact algorithms for multiagent path finding with communication constraints on tree-like structures. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI)*, pages 23177–23185. AAAI Press, 2025. doi:10.1609/AAAI.V39I22.34483.
- 22 Fedor V. Fomin, Petr A. Golovach, Daniel Lokshtanov, Fahad Panolan, Saket Saurabh, and Meirav Zehavi. Multiplicative parameterization above a guarantee. *ACM Trans. Comput. Theory*, 13(3):18:1–18:16, 2021. doi:10.1145/3460956.
- 23 Steven Fortune, John E. Hopcroft, and James Wyllie. The directed subgraph homeomorphism problem. *Theor. Comput. Sci.*, 10:111–121, 1980. doi:10.1016/0304-3975(80)90009-2.
- 24 Shivam Garg and Geevarghese Philip. Raising the bar for vertex cover: Fixed-parameter tractability above a higher guarantee. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1152–1166. SIAM, 2016. doi:10.1137/1.9781611974331.ch80.
- 25 Tzvika Geft and Dan Halperin. Refined hardness of distance-optimal multi-agent path finding. In *AAMAS*, pages 481–488, 2022.

- 592 26 Gregory Z. Gutin and Matthias Mnich. A survey on graph problems parameterized above and
593 below guaranteed values. *CoRR*, abs/2207.12278, 2022. [arXiv:2207.12278](https://arxiv.org/abs/2207.12278), doi:10.48550/
594 [arXiv.2207.12278](https://arxiv.org/abs/2207.12278).
- 595 27 Gregory Z. Gutin, Arash Rafiey, Stefan Szeider, and Anders Yeo. The linear arrangement
596 problem parameterized above guaranteed value. *Theory Comput. Syst.*, 41(3):521–538, 2007.
597 doi:10.1007/s00224-007-1330-6.
- 598 28 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic
599 determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*,
600 4(2):100–107, 1968.
- 601 29 Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven
602 Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Thirty-Fifth AAAI*
603 *Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative*
604 *Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational*
605 *Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pages
606 11272–11281. AAAI Press, 2021. URL: <https://doi.org/10.1609/aaai.v35i13.17344>, doi:
607 10.1609/AAAI.V35I13.17344.
- 608 30 William Lochet. A polynomial time algorithm for the k -disjoint shortest paths problem.
609 In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages
610 169–178. SIAM, 2021. doi:10.1137/1.9781611976465.12.
- 611 31 Daniel Lokshantov, N. S. Narayanaswamy, Venkatesh Raman, M. S. Ramanujan, and Saket
612 Saurabh. Faster parameterized algorithms using linear programming. *ACM Trans. Algorithms*,
613 11(2):15:1–15:31, 2014. doi:10.1145/2566616.
- 614 32 Meena Mahajan, Venkatesh Raman, and Somnath Sikdar. Parameterizing above or below
615 guaranteed values. *J. Computer and System Sciences*, 75(2):137–153, 2009.
- 616 33 Robert Morris, Corina S. Pasareanu, Kasper Søe Luckow, Waqar Malik, Hang Ma, T. K. Satish
617 Kumar, and Sven Koenig. Planning, scheduling and monitoring for airport surface operations.
618 In Daniele Magazzeni, Scott Sanner, and Sylvie Thiébaux, editors, *Planning for Hybrid*
619 *Systems, Papers from the 2016 AAAI Workshop, Phoenix, Arizona, USA, February 13, 2016*,
620 volume WS-16-12 of *AAAI Technical Report*. AAAI Press, 2016. URL: [http://www.aaai.org/](http://www.aaai.org/ocs/index.php/WS/AAAIW16/paper/view/12611)
621 [ocs/index.php/WS/AAAIW16/paper/view/12611](http://www.aaai.org/ocs/index.php/WS/AAAIW16/paper/view/12611).
- 622 34 Christos H. Papadimitriou, Prabhakar Raghavan, Madhu Sudan, and Hisao Tamaki. Motion
623 planning on a graph (extended abstract). In *STOC*, pages 511–520, 1994.
- 624 35 Daniel Ratner and Manfred Warmuth. The $(n^2 - 1)$ -puzzle and related relocation problems.
625 *Journal of Symbolic Computation*, 10(2):111–137, 1990.
- 626 36 Roni Stern, Nathan R. Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne T. Walker,
627 Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Roman Barták, and Eli Boyarski.
628 Multi-agent pathfinding: Definitions, variants, and benchmarks. In *Proceedings of the Twelfth*
629 *International Symposium on Combinatorial Search (SOCS)*, pages 151–158. AAAI Press, 2019.
630 URL: <https://doi.org/10.1609/socs.v10i1.18510>, doi:10.1609/SOCS.V10I1.18510.
- 631 37 Pavel Surynek. An optimization variant of multi-robot path planning is intractable. In
632 *Proceedings of the AAAI conference on artificial intelligence*, volume 24, pages 1261–1263,
633 2010.
- 634 38 Sumanth Varambally, Jiaoyang Li, and Sven Koenig. Which MAPF model works best for
635 automated warehousing? In Lukás Chrpá and Alessandro Saetti, editors, *Proceedings of the*
636 *Fifteenth International Symposium on Combinatorial Search, SOCS 2022, Vienna, Austria,*
637 *July 21-23, 2022*, pages 190–198. AAAI Press, 2022. URL: [https://doi.org/10.1609/socs.](https://doi.org/10.1609/socs.v15i1.21767)
638 [v15i1.21767](https://doi.org/10.1609/socs.v15i1.21767), doi:10.1609/SOCS.V15I1.21767.
- 639 39 Manuela M. Veloso, Joydeep Biswas, Brian Coltin, and Stephanie Rosenthal. Cobots: Robust
640 symbiotic autonomous mobile service robots. In Qiang Yang and Michael J. Wooldridge, editors,
641 *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence,*
642 *IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, page 4423. AAAI Press, 2015. URL:
643 <http://ijcai.org/Abstract/15/656>.

XX:16 Routing Multiple Agents Below the Sum of Distances

- 644 **40** Peter Wurman, Raffaello D'Andrea, and Mick Mountz. Coordinating hundreds of cooperative,
645 autonomous vehicles in warehouses. *AI Magazine*, 29:9–20, 03 2008.
- 646 **41** Jingjin Yu and Steven M. LaValle. Structure and intractability of optimal multi-robot path
647 planning on graphs. In Marie desJardins and Michael L. Littman, editors, *AAAI*. AAAI Press,
648 2013.

Appendix: Full Version of Routing Multiple Agents Below the Sum of Distances

Anonymous author(s)

Anonymous affiliation(s)

Abstract

We study TRANSIENT MULTIAGENT PATHFINDING, a variant of the classical MULTI-AGENT PATHFINDING problem in which a set of agents must be routed without collisions from designated start vertices to designated destination vertices in a graph.

We analyze the problem within the above-and-below-guarantee paradigm of parameterized complexity. In particular, we consider the natural upper bound $\mathcal{L}$, given by the sum of the shortest-path distances between pairs of agents' terminals (corresponding to sequential routing of the agents). The parameterization is given by the gap $\zeta = \mathcal{L} - \lambda$ between this bound and the target makespan λ , together with the number k of agents. Our main result establishes fixed-parameter tractability for the combined parameter $k + \zeta$.

Matching lower bounds show that parameterization by k alone is W[1]-hard and that parameterization by ζ alone is W[1]-hard when terminals are not required to be distinct. On the positive side, if all terminals are distinct, the problem becomes fixed-parameter tractable when parameterized solely by ζ . Finally, we show that TRANSIENT MULTIAGENT PATHFINDING does not admit a polynomial kernel when parameterized by $k + \zeta$. Together, our results provide an almost complete characterization of the parameterized complexity landscape of the problem for the considered parameters.

2012 ACM Subject Classification Mathematics of computing → Combinatorial algorithms; Theory of computation → Fixed parameter tractability

Keywords and phrases Multi-Agent Pathfinding, Parameterized Complexity, Motion Planning

1 Introduction

Gnomes are not known for their sociability. They strongly prefer solitude and take great care never to encounter one another. Unfortunately for them, they must occasionally traverse a shared network of tunnels in order to reach their individual destinations.

We model the tunnel system as a graph, where vertices represent chambers and junctions and edges represent narrow passages between them. Each gnome is assigned a start chamber and a destination chamber. Time advances in discrete steps, and at each step a gnome may either move to an adjacent chamber or remain still. Any encounter between two gnomes is considered disastrous: no two gnomes may occupy the same chamber at the same time, nor may they traverse the same tunnel simultaneously, not even in opposite directions. Once a gnome reaches its destination, it immediately disappears into its private quarters and no longer interacts with the system.

A simple scenario that allows all gnomes to reach their destinations is the following. The gnomes first agree on an order and then move one after another. The first gnome traverses a shortest path to its destination; once it arrives and disappears, the second gnome starts moving, and so on. This straightforward strategy is easy to compute and yields a schedule whose makespan, that is, the total time needed to move the gnomes to their destinations, is upper bounded by 1 plus the sum of the lengths of the shortest paths of all gnomes. Achieving even a slightly smarter strategy quickly becomes nontrivial: Avoiding encounters may require gnomes to wait inside chambers, to take detours, or to carefully interleave their movements.



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

The main question we address in this work is whether it is possible to obtain a schedule with a smaller makespan and how difficult this task is computationally.

Of course, while the life of a gnome is certainly an interesting subject, the main question we address arises from its connections to more practical, real-life problems.

Routing multiple agents from designated start vertices to designated destination vertices without collisions is a fundamental problem in multi-agent planning. The problem is strongly motivated by numerous real-world applications, including automated warehouse systems [29, 40], traffic management and control [33], and robotics [39]. A well-known example is the coordination of large fleets of mobile robots in Amazon fulfillment centers, where systems such as Kiva rely on efficient and reliable multi-robot path planning to transport inventory pods while avoiding collisions [40]. From a theoretical perspective, almost all variants of the problem are NP-hard [15, 35, 41].

The range of applications such as automated warehouses, aircraft towing, UAV traffic management, or drone delivery has led to the development of numerous problem variants (see, e.g., [36, 38] for surveys).

The problem is commonly formalized as MULTIAGENT PATHFINDING (MAPF), where the environment is modeled as an n -vertex graph and each of the k agents has a specified start vertex s_i and goal vertex t_i . At each discrete time step, an agent either waits or moves along an edge to a neighboring vertex; no vertex or edge may be used by more than one agent in the same time step. In the standard formulation of MAPF, the objective is to find a schedule that routes all agents to their goals while minimizing the *makespan* λ (the arrival time of the last agent) [3, 12, 13, 18, 20]. Another popular measure is the total distance traveled by the agents [10, 11, 25, 34]. Let us remark that the total distance travel by the agents is always at least the sum of the lengths of the shortest paths between all terminals.

We focus on algorithms with provable guarantees for computing schedules of optimal makespan. The problem has been extensively studied and is computationally intractable in the classical sense: it is NP-hard not only on general graphs, but also on solid grid graphs [13] and even on trees [20]. Given this intractability, numerous heuristic and algorithmic approaches have been proposed, including SAT-based methods [37], and A^* -based algorithms [28]; see also the surveys [36, 38].

There are two common assumptions regarding how agents behave upon reaching their targets; see [36]. Under the first assumption, an agent remains at its target vertex until all agents have reached their respective targets. Under the second assumption, agents disappear immediately after reaching their targets. Our results concern the latter assumption. We adopt the model in which agents enter the graph at their start times at s_i , traverse along edges, and immediately disappear upon reaching t_i ; in particular, t_i is not occupied after arrival.

In this work, we adopt the framework of parameterized complexity [17, 9]. Although the computational complexity of many fundamental coordinated motion planning variants was established several decades ago [34], recent years have witnessed renewed interest in these problems through the lens of parameterized complexity [4, 12, 14, 18, 19, 20, 21].

Most of this line of work focuses on natural parameterizations such as the number of agents, the makespan, or structural parameters of the input graph, for example its treewidth. We take a different perspective here. In particular, we follow the above-or-below-guarantee paradigm, a popular approach in parameterized complexity. The term above (below) guarantee parameterization was introduced by Mahajan and Raman [32]. This

concept has proved fruitful for studying numerous problems in parameterized complexity [1, 6, 8, 22, 24, 27, 31], see also the survey [26].

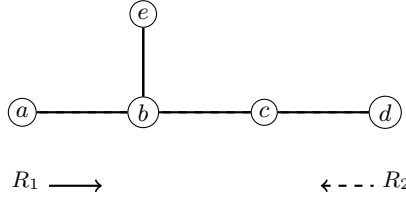
Consider the simple sequential protocol that routes the agents one after another along shortest s_i - t_i -paths. It is not difficult to show (see Lemma 3) that this protocol guarantees completion within $\lambda \leq \mathcal{L}_{\mathcal{R},G} := 1 + \sum_{i \in [k]} \text{dist}_G(s_i, t_i)$ time steps.

This bound is tight: there exist instances of the problem that require exactly $\mathcal{L}_{\mathcal{R},G}$ rounds. For example, let G be a path with end-vertices u and v , and consider two agents with terminals $s_1 = t_2 = u$ and $s_2 = t_1 = v$. In this case, neither agent can start moving before the other has reached its destination and disappeared; otherwise, they would block each other along the path. Consequently, one agent must wait until the other completes its traversal, incurring at least one additional round between their movements. Therefore, any feasible schedule has makespan at least $1 + \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) = \mathcal{L}_{\mathcal{R},G}$. Hence, the parameter $\zeta_{\mathcal{R},G} = \mathcal{L}_{\mathcal{R},G} - \lambda$ quantifies how much tighter a schedule we obtain compared to the (sometimes pessimistic) upper bound $\mathcal{L}_{\mathcal{R},G}$. In the terminology of parameterized complexity, parameterizing by $\zeta_{\mathcal{R},G}$ corresponds to *parameterization below the trivial upper bound* $\mathcal{L}_{\mathcal{R},G}$.

Before proceeding with an overview of our results, let us give an example of a possible scheduling of the agents, postponing the formal definitions to Section 2.

Example

Consider the following graph with two agents R_1 and R_2 , where R_1 needs to move from a to d and R_2 from d to a .



The upper bound $\mathcal{L}_{\mathcal{R},G}$ in this case is

$$1 + \sum_{i \in [2]} \text{dist}_G(s_i, t_i) = 1 + \text{dist}_G(a, d) + \text{dist}_G(d, a) = 1 + 3 + 3 = 7.$$

If both agents traverse the shortest path a - b - c - d starting at time 0 in opposite directions, then at some time step they traverse the same edge in opposite directions. These routes are *conflicting*. A valid (but not optimal) schedule is when first R_1 moves from a to d , and then R_2 moves from d to a . That is,

time step x	R_1	R_2
0	a	—
1	b	—
2	c	—
3	d (disappears)	—
4	—	d
5	—	c
6	—	b
7	—	a (disappears)

The makespan λ in this case is equal to 7, the moment agent R_2 reaches a . It is also equal to the upper bound $\mathcal{L}_{\mathcal{R},G}$.

XX:4 Routing Multiple Agents Below the Sum of Distances

Consider another, now optimal, scheduling, where R_1 use the side vertex e as a buffer:

time step x	R_1	R_2
0	a	d
1	b	c
2	e	b
3	b	a (disappears)
4	c	—
5	d (disappears)	—

At no time step do the agents occupy the same vertex or traverse the same edge simultaneously. Hence, the routes are non-conflicting. Agent R_1 finishes at time 5, agent R_2 at time 3, so the makespan is $\lambda = 5$, improving over the upper $\mathcal{L}_{\mathcal{R},G}$ by $\zeta_{\mathcal{R},G} = \mathcal{L}_{\mathcal{R},G} - \lambda = 2$.

Our contribution. We consider the following problem.

TRANSIENT MULTIAGENT PATHFINDING

Input: A connected graph G , integers k and λ , and a set $\mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}$ of agents.

Task: Is there a schedule for $\mathcal{R}$ of makespan at most λ ?

Our main result shows that the problem is fixed-parameter tractable with respect to the combined parameter $k + \zeta$, where k is the number of agents and ζ measures the improvement over the upper bound $\mathcal{L}$.

► **Theorem 1.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with $\lambda = \mathcal{L}_{\mathcal{R},G} - \zeta$.*

Overview of the proof of Theorem 1. The proof is based on structural properties of shortest paths. The starting point is the fact (see Lemma 7) that TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k\lambda)} \cdot n^{\mathcal{O}(1)}$ time. Thus, Theorem 1 holds if $\lambda = \mathcal{L}_{\mathcal{R},G} - \zeta = \mathcal{O}(k\zeta)$. In particular, this is fulfilled if $\max_{i \in [k]} \text{dist}_G(s_i, t_i) = \mathcal{O}(\zeta)$. We prove that solving TRANSIENT MULTIAGENT PATHFINDING can be reduced to this special case when the length of shortest s_i - t_i -paths is $\mathcal{O}(\zeta)$. For this, we consider an instance where there is $i \in [k]$ such that the distance between s_i and t_i is sufficiently large, and either demonstrate that this is a no-instance or construct a routing with makespan at most λ . Moreover, to rule out the no-instances, it is sufficient to verify whether there is $i \in [k]$ with $\text{dist}_G(s_i, t_i) > \lambda$ or there are distinct $i, j \in [k]$ such that the agents R_i and R_j cannot be routed within the makespan λ . The latter can be checked by an XP algorithm solving TRANSIENT MULTIAGENT PATHFINDING in $(kn)^{\mathcal{O}(k)}$ time (see Lemma 6). In all other cases, we are able to construct a routing schedule of the makespan at most λ .

We briefly sketch the main ideas behind constructing a routing schedule in the case when there is a pair of faraway terminals. The proof is technical and is obtained by a sequence of lemmas establishing sufficient conditions for the existence of a feasible schedule.

Given two terminal pairs (s_i, t_i) and (s_j, t_j) , we consider *laminar* shortest s_i - t_i - and s_j - t_j -paths P_i and P_j , respectively, that is, shortest paths whose intersection is either empty or a path. We distinguish *codirectional* and *opposite* paths. Paths P_i and P_j are codirectional if the agents transverse the common subpath in the same direction following P_i and P_j , and the paths are opposite, otherwise (see Figure 2). We note that the agents can follow codirectional shortest paths almost simultaneously. This implies that if there are codirectional

Parameter	Result
$k + \zeta$	FPT in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ (Theorem 1); no polynomial kernel (Theorem 26)
k	W[1]-hard (subcubic graphs) (Corollary 24); In XP (Lemma 6)
ζ	W[1]-hard (Theorem 25); Para-NP vs. XP is open

■ **Table 1** Parameterized complexity of TRANSIENT MULTIAGENT PATHFINDING.

shortest path P_i and P_j of length $\Omega(\zeta)$, then we can route the agents R_i and R_j saving considerable time, and this is sufficient for the existence of a routing of the makespan at most λ . We also observe that in some cases, opposite paths of length $\Omega(\zeta)$ can be used to save time. In particular, this happens if the common part of the paths is sufficiently far from one of the end-points of the paths.

If the above cases do not apply, we observe that if there are three terminal pairs with terminals at distance $\Omega(\zeta)$, we can find codirectional shortest paths for two pairs. Thus, the analysis boils down to the cases when the terminals of either one or two pairs are at distance at least $c \cdot \zeta$ for a specific constant $c > 0$, and for all other pairs (s_i, t_i) , the distance between s_i and t_i is upper bounded by $c' \cdot \zeta$ for another constant $c' < c$. If there is exactly one shortest path of length at least $c \cdot \zeta$, then we can route the corresponding agent R_i along a shortest path. Simultaneously with R_i , we can route other agents and save time.

The case of two pairs of faraway terminals is the most challenging. Here, we need some additional structural results about paths in G . Very roughly, the idea is to route two agents R_i and R_j with $\text{dist}_G(s_i, t_i), \text{dist}_G(s_j, t_j) \geq c \cdot \zeta$ in the optimum way (which is proved to be very special after excluding some cases), and route the other agents while R_i and R_j follow their routes. This concludes the proof overview.

Of course, a natural question is whether Theorem 1 is tight. In other words, whether the combined parameterization is necessary. We complement our theorem with lower bounds showing that this is indeed the case. First, we observe (Theorem 23 and Corollary 24) that TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard when parameterized by k , even if the input graph is subcubic. We also prove (Theorem 25) that TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard when parameterized by ζ . Therefore, fixed-parameter tractability with respect to only one of these parameters is highly unlikely. Regarding single-parameter parameterizations, we observe (Lemma 6) that for fixed k , TRANSIENT MULTIAGENT PATHFINDING is solvable in polynomial time $(kn)^{\mathcal{O}(k)}$. That is, it belongs to XP when parameterized by k . Whether the problem is Para-NP-complete or in XP when parameterized by ζ remains an interesting open question.

Having established an FPT result, a natural next question concerns the existence of a polynomial kernel. We show in Theorem 26 that TRANSIENT MULTIAGENT PATHFINDING parameterized by $k + \zeta$ does not admit a polynomial kernel. Combined with the above lower bounds, Theorem 1 yields an almost complete characterization of the parameterized complexity of TRANSIENT MULTIAGENT PATHFINDING. We summarize these results in Table 1.

Although our lower bounds exclude the existence of an FPT algorithm for TRANSIENT MULTIAGENT PATHFINDING parameterized solely by k , we identify a natural subclass that does admit such algorithms. In particular, when all terminals are distinct, the upper bound on λ improves to $\mathcal{L}_{\mathcal{R},G}^* = \sum_{i \in [k]} \text{dist}(s_i, t_i)$. We establish the following theorem.

► **Theorem 2.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(\zeta^3)} \cdot n^{\mathcal{O}(1)}$*

XX:6 Routing Multiple Agents Below the Sum of Distances

Symbol	Meaning
$\mathbb{N}$	Non-negative integers.
G	Undirected input graph.
$V(G), E(G)$	Vertex and edge sets of G .
$\text{dist}_G(a, b)$	Distance (the number of edges in the shortest path) between a and b in G .
k	Number of agents.
$\mathcal{R} = \{R_1, \dots, R_k\}$	Set of agents.
$R_i = (s_i, t_i)$	Agent i with start s_i and destination t_i .
s_i, t_i	Start/destination of R_i ($s_i \neq t_i$).
$\{s_i \mid i \in [k]\} \cup \{t_i \mid i \in [k]\}$	Set of terminals.
(W_i, x_i)	Route of R_i .
$W_i = (u_0, \dots, u_{\delta_i})$	Positions of R_i along its route.
x_i	Moment when R_i starts moving along W_i .
δ_i	Length of W_i (steps).
u_y	Position of R_i at time $x_i + y$.
free/occ.	Vertex status at a time step (no/some agent at v).
Γ	A schedule: pairwise non-conflicting routes.
λ	Makespan of Γ ($\max_i(x_i + \delta_i)$).
$\mathcal{L}_{\mathcal{R}, G}$ (or $\mathcal{L}$)	$1 + \sum_{i \in [k]} \text{dist}_G(s_i, t_i)$, the upper bound on the makespan.
$\zeta_{\mathcal{R}, G}$ (or ζ)	$\mathcal{L}_{\mathcal{R}, G} - \lambda$, the gain below the upper bound.

■ **Table 2** Summary of notation.

time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with pairwise distinct terminals and $\lambda = \mathcal{L}_{\mathcal{R}, G}^* - \zeta$.

2 Preliminaries

We use $\mathbb{N}$ to denote the set of non-negative integers, and write $[p]$ and $[0, p]$ for the set $\{1, \dots, p\}$ and $\{0, \dots, p\}$, respectively, where $p \in \mathbb{N}$. We refer to the textbook [16] for the basic graph-theoretic notions and notation. The sets of vertices of a graph G are denoted by $V(G)$ and $E(G)$, respectively. We write $P = v_0, \dots, v_\ell$, where $v_0, \dots, v_\ell \in V(G)$ are distinct vertices of G such that $v_{i-1}v_i \in E(G)$ for all $i \in [\ell]$, to denote a *path* of length ℓ in G . The vertices v_0 and v_ℓ are the *end-vertices* of P , and $v_1, \dots, v_{\ell-1}$ are *internal*. For a path P with end-vertices s and t , we say that P is an *s-t-path*. For an undirected graph G and two vertices $u, v \in V(G)$, we denote by $\text{dist}_G(u, v)$ the distance, i.e., the length of the shortest path, between u and v in G . We drop the subscript, if the graph G is clear from the context. Throughout the paper, we use n to denote the number of vertices of a graph if it does not create confusion.

We are given an undirected graph G and a set $\mathcal{R} = \{R_1, R_2, \dots, R_k\}$ of k agents. Each $R_i \in \mathcal{R}$ has a starting vertex s_i and a destination vertex t_i in $V(G)$ such that $s_i \neq t_i$. We refer to the elements in the set $\{s_i \mid i \in [k]\} \cup \{t_i \mid i \in [k]\}$ as *terminals*. We use a discrete time frame $[0, t]$, $t \in \mathbb{N}$, to reference the sequence of moves of the agents, and in each time step $x \in [0, t]$, every agent $R_i \in \mathcal{R}$ performs one of the following actions:

- R_i enters the graph at s_i ;
- R_i moves from its current position $v \in V(G)$ to a neighbor of v ;
- R_i stays at its current position $v \in V(G)$; or
- R_i disappears from the graph if its current position is t_i .

A *route* for agent R_i is a pair (W_i, x_i) , where W_i is a sequence $(u_0, \dots, u_{\delta_i})$ of vertices in G and such that

- u_0 is the starting vertex s_i of R_i ;

259 ■ u_{δ_i} is the destination vertex t_i of R_i ;
 260 ■ for each $y \in [\delta_i]$ it holds that $u_{y-1} = u_y$ or $u_{y-1}u_y$ is an edge in G ,
 261 and $x_i \in \mathbb{N}$. The vertex u_y is the position of R_i at time step $x_i + y$. Thus, R_i appears at
 262 moment x_i in s_i and moves along W_i from s_i to t_i ; the agent disappears when it reaches t_i .
 263 A vertex $v \in V(G)$ is *free* at time step $x \in [0, t]$ if no agent is located at v at time step x ;
 264 otherwise, v is *occupied*.

265 For $1 \leq i < j \leq k$, two routes $(W_i = (u_0, \dots, u_{\delta_i}), x_i)$ and $(W_j = (v_0, \dots, v_{\delta_j}), x_j)$
 266 are *conflicting* (or *in conflict*) if there exists some time step x such that R_i and R_j are
 267 either at the same vertex or traversing the same edge at the time step x . More formally,
 268 $x = x_i + y = x_j + z$ such that either $u_y = v_z$ or $u_{y-1}u_y = v_{z-1}v_z$. A *schedule* Γ for $\mathcal{R}$ is a
 269 set of pairwise non-conflicting routes $\{(W_i, x_i) \mid i \in [k]\}$, during a time interval $[0, \lambda]$ (that
 270 is, $\max_{i \in [k]}(x_i + \delta_i) = \lambda$). The integer λ is called the *makespan* of Γ . Using the introduced
 271 terminology, we formalize TRANSIENT MULTIAGENT PATHFINDING stated in Section 1. The
 272 above notation is summarized in Table 2.

273 For the introduction to Parameterized Complexity, we refer to the textbooks [9, 17]. Here,
 274 we just informally remind some basic notions. A *parameterized problem* $L \subseteq \Sigma \times \mathbb{N}$, where Σ^*
 275 is the set of strings over a finite alphabet Σ . Respectively, its instance is a pair (x, k) ,
 276 where $x \in \Sigma^*$ and $k \in \mathbb{N}$ is a *parameter*. A parameterized problem L is *fixed-parameter*
 277 *tractable* (FPT) if there exists an algorithm solving L in $f(k) \cdot |x|^{\mathcal{O}(1)}$ time, where f is a
 278 computable function. Furthermore, L is *slicewise polynomial* (XP) if L can be solved in $|x|^{g(k)}$
 279 time for a computable function g . The classes FPT and XP consist of the fixed-parameter
 280 tractable and slicewise polynomial parameterized problems, respectively. The standard
 281 way to exclude the existence of an FPT algorithm for a parameterized problem (up to
 282 reasonable complexity assumptions) is to show its hardness for the class W[1] (see [9, 17] for
 283 the definitions) using a *parameterized reduction*, that is, a reduction running in $f(k) \cdot |x|^{\mathcal{O}(1)}$
 284 time for a computable f .

285 A *kernelization* (or *kernel*) for a parameterized problem L is a polynomial-time algorithm
 286 that for any instance (x, k) outputs an instance (x', k') of L such that (i) $(x, k) \in L$ if
 287 and only if $(x', k') \in L$ and (ii) $|x'| + k' \leq f(k)$ for a computable function f . A kernel is
 288 *polynomial* if f is polynomial. While a decidable parameterized problem is in FPT if and
 289 only if it admits a kernel, it is unlikely that all fixed-parameter tractable parameterized
 290 problems admit polynomial kernels. In particular, it is standard to exclude the existence of
 291 a polynomial kernel up to the assumption that $\text{coNP} \not\subseteq \text{NP/poly}$ by providing a *polynomial*
 292 *parameter transformation*, that is, a parametrized reduction where f is polynomial, from the
 293 parameterized problem for which such a kernelization lower bound is already established.

294 3 Basic observations

295 In this section, we collect basic observations on TRANSIENT MULTIAGENT PATHFINDING.
 296 First, we formally prove our upper bound for the minimum makespan and show some useful
 297 properties of schedules guaranteeing such a makespan.

298 ► **Lemma 3.** *Let $(G, k, \mathcal{R}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING. If*
 299 *$\lambda \geq 1 + \sum_{i \in [k]} \text{dist}(s_i, t_i)$, then $(G, k, \mathcal{R}, \lambda)$ is a yes-instance. Moreover, there is a schedule*
 300 *of makespan at most λ such that for every $i \in [k]$,*
 301 *(i) the route of R_i is (P_i, x_i) where P_i is an arbitrary shortest s_i - t_i -path, and*
 302 *(ii) for each internal vertex v of P_i and the moment at which R_i is in v , R_i is an only agent*
 303 *on G .*

Proof. We assume that the agents $R_1, \dots, R_k$ are ordered in such a way that $s_i \neq t_{i-1}$ holds for all $i \in [2, \ell]$, with $\ell \leq k$ chosen as large as possible. We navigate the agents $R_1, \dots, R_\ell$ sequentially on shortest paths, such that we start the route of the first agent at the moment 0 and the route of agent $i \geq 2$ exactly at the same time when the $(i-1)$ -th agent reaches its destination (that is at the time step $\sum_{j \in [i-1]} \text{dist}_G(s_j, t_j)$). It is straightforward that the routes in this (partial) schedule have no conflicts. If $\ell = k$ then we have the schedule with the makespan $\sum_{i \in [k]} \text{dist}_G(s_i, t_i)$. Assume that this is not the case and $\ell < k$.

By the choice of ℓ , we have that for every $i \in [\ell+1, k]$, $s_i = t_\ell$ as, otherwise, we would be able to extend the ordering $R_1, \dots, R_\ell$ by appending one of the remaining agents. We also have that for every $i \in [\ell+1, k]$, $t_i = s_1$ because we would be able to insert the agent with $t_i \neq s_1$ for $i \in [\ell+1, k]$ in the beginning of the ordering, contradicting the choice of ℓ . Note that this implies that $s_1 \neq t_\ell$ as $s_i \neq t_i$ for all $i \in [k]$. Then, we can extend the partial schedule already constructed for $R_1, \dots, R_\ell$ as follows. We set $t = \sum_{i \in [\ell]} \text{dist}_G(s_i, t_i) + 1$, and then we navigate the agents $R_\ell, \dots, R_k$ using the same strategy as for the previous agent starting from the moment t . More precisely, for each $i \in [\ell+1, k]$ we move R_i from s_i to t_i along the shortest path, and R_i starts moving at the moment $1 + \sum_{j \in [i-1]} \text{dist}_G(s_j, t_j)$. Because $s_i = t_\ell$ and $t_i = s_1 \neq s_i$ for each $i \in [\ell+1, k]$ and R_ℓ starts to move at the moment $t + 1$, the constructed schedule is feasible, that is, the routes have no conflicts. Because the makespan is $1 + \sum_{i \in [k]} \text{dist}_G(s_i, t_i)$, we have that $(G, k, \mathcal{R}, \lambda)$ is a yes-instance if $\lambda \geq 1 + \sum_{i \in [k]} \text{dist}_G(s_i, t_i)$. It is straightforward to see that the constructed schedule satisfies requirements (i) and (ii). This concludes the proof. $\blacktriangleleft$

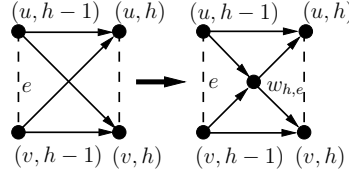
We note that the bound in Lemma 3 is tight. Let G be a path with end-vertices u and v . We define $s_1 = t_2 = u$ and $s_2 = t_1 = v$. Then routing two agents $R_1 = (s_1, t_1)$ and $R_2 = (s_2, t_2)$ demands $\text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) + 1$ steps. From now on, let us denote by $\mathcal{L}_{\mathcal{R}, G}$ (or just $\mathcal{L}$ if G and $\mathcal{R}$ are clear from the context) the value $\mathcal{L}_{\mathcal{R}, G} = 1 + \sum_{i \in [k]} \text{dist}(s_i, t_i)$. Similarly, we denote by $\zeta_{\mathcal{R}, G}$ (or just ζ) the value $\zeta_{\mathcal{R}, G} = \mathcal{L}_{\mathcal{R}, G} - \lambda$.

In Theorem 1, we exploit the facts that TRANSIENT MULTIAGENT PATHFINDING is in XP when parameterized by k and in FPT for the parameterization by k and λ . To show these properties, we reduce the problem to finding paths in auxiliary time-expansion graphs.

Let G be a graph and let $\lambda \geq 0$ be an integer. We denote by $\mathcal{G}_\lambda$ the directed *time-expansion* graph with the vertex set $\{(v, t) \mid v \in V(G) \text{ and } t \in [0, \lambda]\}$ where there is an arc from a vertex (u, i) to a vertex (v, j) if and only if $j - i = 1$ and $v \in N_G[u]$. In other words, the set of vertices consists of $\lambda + 1$ copies of $V(G)$ forming layers, arcs join vertices of subsequent layers, and (u, i) from the i -th layer is joined with arcs with the copies of the vertices v of G in the $(i+1)$ -th layer that are either the same as u or adjacent to u in G . Notice that $\mathcal{G}_\lambda$ is a directed acyclic graph (DAG). The construction of $\mathcal{G}_\lambda$ and the definition of the routes in TRANSIENT MULTIAGENT PATHFINDING imply the following observation.

► Observation 4. *An instance $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ of TRANSIENT MULTIAGENT PATHFINDING is a yes-instance if and only if there are integers $p_1, \dots, p_k \in [0, \lambda]$ and $q_1, \dots, q_k \in [0, \lambda]$ such that $\mathcal{G}_\lambda$ has vertex disjoint directed paths $P_1, \dots, P_k$, where P_i is an (s_i, p_i) - (t_i, q_i) -path for every $i \in [k]$, with the additional property that for every $h \in [\lambda]$ and each edge $\{u, v\}$ of G , the arcs $((u, h-1), (v, h))$ and $((v, h-1), (u, h))$ are not used by two paths.*

Furthermore, we can augment $\mathcal{G}_\lambda$ by artificial terminals and modify the graph to encode the additional property to reduce TRANSIENT MULTIAGENT PATHFINDING to the DISJOINT PATHS problem. We remind the reader that the task of DISJOINT PATHS is, given a (directed) graph G and k pairs of terminal vertices (s_i, t_i) for $i \in [k]$, to decide whether there are k



347 **Figure 1** Arc modifications in the construction of $\mathcal{G}_\lambda^*$.

352 vertex disjoint paths joining these pairs of terminals. Let $s_i, t_i \in V(G)$ be terminal vertices
 353 for $i \in [k]$. We construct $\mathcal{G}_\lambda^*$ as follows:

- 354 ■ construct $\mathcal{G}_\lambda$ and $2k$ vertices s_i^* and t_i^* for $i \in [k]$,
- 355 ■ for each $i \in [k]$ and every $j \in [0, \lambda]$, construct arcs $(s_i^*, (s_i, j))$ and $((t_i, j), t_i^*)$,
- 356 ■ for each $h \in [\lambda]$ and every edge $e = \{u, v\}$ of G , construct a vertex $w_{h,e}$ and replace the arcs
 357 $((u, h-1), (v, h))$ and $((v, h-1), (u, h))$ by four arcs $((u, h-1), w_{h,e})$, and $(w_{h,e}, (v, h))$,
 358 $((v, h-1), w_{h,e})$, $(w_{h,e}, (u, h))$;

359 see Figure 1 for the arc modifications in the construction. Notice that $\mathcal{G}_\lambda^*$ is a DAG. By
 360 Observation 4, we obtain the following claim.

361 ► **Observation 5.** *An instance $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ of TRANSIENT
 362 MULTIAGENT PATHFINDING is a yes-instance if and only if $\mathcal{G}_\lambda^*$ has vertex disjoint directed
 363 paths $P_1, \dots, P_k$, where P_i is an $s_i^*-t_i^*$ -path for every $i \in [k]$.*

364 **Proof.** To see the equivalence between the conditions in Observation 4 and Observation 5,
 365 note that for vertex disjoint paths s_i-t_i -paths in $\mathcal{G}_\lambda^*$, it can be assumed that they are induced.
 366 Then for every $h \in [\lambda]$ and each $e = \{u, v\} \in E(G)$, $((v, h-1), w_{h,e}, (v, h))$ is not a subpath of
 367 any path. Otherwise, we can replace $((v, h-1), w_{h,e}, (v, h))$ by $((v, h-1), (v, h))$. To conclude
 368 the proof, it is sufficient to observe that for every $h \in [\lambda]$ and each $e = \{u, v\} \in E(G)$,
 369 $((u, h-1), w_{h,e}, (v, h))$ and $((v, h-1), w_{h,e}, (u, h))$ are not disjoint and, therefore, cannot be
 370 subpaths of vertex disjoint paths in $\mathcal{G}_\lambda^*$. ◀

371 Because DISJOINT PATHS can be solved in $n^{\mathcal{O}(k)}$ time for k pairs of terminals on DAGs
 372 by the result of Fortune, Hopcroft, and Wyllie [23], by Observation 5, we have the following.

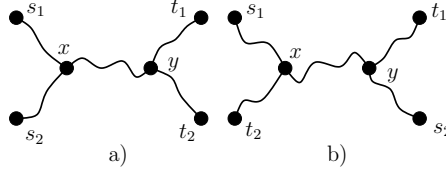
373 ► **Lemma 6.** *TRANSIENT MULTIAGENT PATHFINDING is solvable in time $(kn)^{\mathcal{O}(k)}$.*

374 **Proof.** Let $I = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT
 375 MULTIAGENT PATHFINDING where G is a graph with n vertices and m edges. If $\lambda \geq \mathcal{L}_{\mathcal{R}, G}$,
 376 then I is a yes-instance by Lemma 3. Otherwise, we construct $\mathcal{G}_\lambda^*$. As the distance between
 377 any pair of terminals is at most $n-1$, $\lambda \leq (n-1)k$. Hence, we have that $\mathcal{G}_\lambda^*$ has at
 378 most $(\lambda+1)n + m\lambda + 2k = ((n-1)k+1)n + m(n-1)k + 2k$ vertices. Then, we call the
 379 algorithm by Fortune et al. [23] to solve DISJOINT PATH on $\mathcal{G}_\lambda^*$ with the pairs (s_i^*, t_i^*)
 380 for $i \in [k]$ of terminals in $(nk)^{\mathcal{O}(k)}$ time. By Observation 5, solving DISJOINT PATHS for the
 381 constructed instance is equivalent to solving TRANSIENT MULTIAGENT PATHFINDING for I .
 382 This concludes the proof. ◀

383 Observation 5 also allows us to show that TRANSIENT MULTIAGENT PATHFINDING is
 384 FPT when parameterized by k and λ by making use of the *color coding* technique of Alon,
 385 Yuster, and Zwick [2].

386 ► **Lemma 7.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k\lambda)} \cdot n^{\mathcal{O}(1)}$ time.*

XX:10 Routing Multiple Agents Below the Sum of Distances



406 **Figure 2** Laminar codirectional (a) and opposite (b) s_1-t_1 and s_2-t_2 -paths.

387 **Proof.** The color coding technique can be used directly but, for simplicity, we reduce
 388 TRANSIENT MULTIAGENT PATHFINDING to the variant of DISJOINT PATHS where each path
 389 should have a prescribed length. This variant belongs to the class of problems solvable by
 390 color coding [2] (see also the textbook [9]). By Observation 5, TRANSIENT MULTIAGENT
 391 PATHFINDING is equivalent to DISJOINT PATHS on $\mathcal{G}_\lambda^*$ with the terminal pairs (s_i^*, t_i^*) for $i \in [k]$.
 392 Notice that for any $i \in [k]$, the length of an $s_i^*-t_i^*$ -path is at least 4 and at most $2\lambda + 2$.
 393 We guess the length $\ell_i \leq 2\lambda + 2$ of the $s_i^*-t_i^*$ -path in a (potential) solution. For each of
 394 at most $(2\lambda)^k$ guesses, we use the color coding algorithm from [2] to check whether $\mathcal{G}_\lambda^*$
 395 has vertex disjoint $s_i^*-t_i^*$ -paths P_i where the length of P_i is exactly ℓ_i . This can be done
 396 in $2^{\mathcal{O}(\sum_{i=1}^k (\ell_i+1))} \cdot n^{\mathcal{O}(1)}$ time. If we find a solution for one of the guesses, we conclude
 397 that the considered instance of TRANSIENT MULTIAGENT PATHFINDING is a yes-instance.
 398 Otherwise, if we fail to find a solution for all guesses, we conclude that we have a no-instance.
 399 As we have at most $(2\lambda)^k$ guesses and $\sum_{i=1}^k (\ell_i + 1) \leq (2\lambda + 3)k$, we obtain that the overall
 400 running time is $2^{\mathcal{O}(k\lambda)} \cdot n^{\mathcal{O}(1)}$, concluding the proof. $\blacktriangleleft$

401 **4 Transient Multiagent Pathfinding parameterized below $\mathcal{L}$**

402 In this section, we prove our main result, Theorem 1: TRANSIENT MULTIAGENT PATHFINDING
 403 is FPT when parameterized by k and $\zeta = \mathcal{L}_{\mathcal{R},G} - \lambda$. Throughout the section, we assume that
 404 $\lambda \leq \mathcal{L}_{\mathcal{R},G}$ in the considered instances (otherwise, we have a trivial yes-instance by Lemma 3).
 405 We start with some notation and auxiliary results.

407 Let (s_1, t_1) and (s_2, t_2) be pairs of distinct vertices of a graph G . We say that an
 408 s_1-t_1 -path P_1 and s_2-t_2 -path P_2 are *laminar* if either they have no common vertices or
 409 the intersection of P_1 and P_2 is a path in G (see Figure 2). Laminar paths P_1 and P_2
 410 are *codirectional* if either they are disjoint or their intersection is an x - y -path Q where
 411 either $\text{dist}_G(s_1, x) \leq \text{dist}_G(s_1, y)$ and $\text{dist}_G(s_2, x) \leq \text{dist}_G(s_2, y)$ or $\text{dist}_G(s_1, y) \leq \text{dist}_G(s_1, x)$
 412 and $\text{dist}_G(s_2, y) \leq \text{dist}_G(s_2, x)$ (see Figure 2(a)). Otherwise, we say that P_1 and P_2 are
 413 *opposite* (see Figure 2(b)). We make the following easy observation.

414 **► Observation 8.** For any two pairs (s_1, t_1) and (s_2, t_2) of distinct vertices of a graph G ,
 415 G has laminar shortest s_1-t_1 - and s_2-t_2 -paths. Moreover, for any given shortest s_1-t_1 and
 416 s_2-t_2 -paths P_1 and P_2 , there is a shortest s_1-t_1 -path P'_1 such that P'_1 and P_2 are laminar.

417 **Proof.** Consider arbitrary shortest s_1-t_1 and s_2-t_2 -paths P_1 and P_2 . If the paths are disjoint
 418 then they are laminar by definition. Otherwise, let x and y be the vertices of $V(P_1) \cap V(P_2)$
 419 such that the vertices of the s_1-x and $y-t_2$ -subpaths of P_1 (except for x and y) are not in
 420 P_2 ; note that it may happen that $x = s_1$ or $y = t_2$. Consider the s_1-t_1 -path P'_1 obtained by
 421 concatenating the s_1-x -subpath of P_1 , the $x-y$ -subpath of P_2 , and the $y-t_1$ -subpath of P_1 .
 422 Because every subpath of a shortest path is a shortest path, we have that P'_1 is a shortest
 423 s_1-t_1 -path. By construction, P'_1 and P_2 are laminar. This concludes the proof. $\blacktriangleleft$

In the series of the next lemmas, we give sufficient conditions ensuring that an instance of TRANSIENT MULTIAGENT PATHFINDING is a yes-instance. The first condition is the existence of two codirectional laminar shortest paths of length $\Omega(\zeta)$.

► **Lemma 9.** *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Suppose that there are distinct $i, j \in [k]$ such that*

- (i) $\text{dist}_G(s_i, t_i) \geq \zeta + 2$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 2$,
 - (ii) *there are codirectional laminar shortest s_i - t_i and s_j - t_j -paths.*
- Then, $\mathcal{I}$ is a yes-instance.

Proof. Let P_i and P_j be codirectional laminar shortest s_i - t_i and s_j - t_j -paths, and let $p = \text{dist}_G(s_i, t_i)$, $q = \text{dist}_G(s_j, t_j)$.

Assume first that P_i and P_j are disjoint. We assume without loss of generality that $p \leq q$. We define the routes of R_i and R_j as $(P_i, 0)$ and $(P_j, 0)$, respectively, that is, these agents move to their destinations along the shortest paths starting at the moment 0. Since P_i and P_j are disjoint, the routes are not in conflict. They reach the destinations by the moment q . We route the remaining agents starting at the moment $q + 1$. By Lemma 3, they can be routed within the makespan at most $\sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1$. Thus, we obtain a schedule with the total makespan at most

$$\begin{aligned}
 & q + 1 + \sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1 \\
 &= \sum_{h \in [k] \setminus \{i\}} \text{dist}_G(s_h, t_h) + 2 \\
 &= \sum_{h \in [k]} \text{dist}_G(s_h, t_h) + 2 - p \\
 &= \mathcal{L}_{\mathcal{R}, G} + 1 - p.
 \end{aligned}$$

Since $p \geq \zeta + 2$, we have that the makespan of the schedule is at most $\mathcal{L}_{\mathcal{R}, G} - \zeta$. This means that $\mathcal{I}$ is a yes-instance.

Suppose from now on that P_i and P_j have common vertices. Because P_i and P_j are laminar, their intersection is a path Q . Let x and y be the end-vertices of Q . Since the paths are codirectional, we can assume that $\text{dist}_G(s_i, x) \leq \text{dist}_G(s_i, y)$ and $\text{dist}_G(s_j, x) \leq \text{dist}_G(s_j, y)$, that is, x occurs first on the paths. Furthermore, we assume without loss of generality that $\text{dist}_G(s_i, x) \leq \text{dist}_G(s_j, x)$. We define the routes of R_i and R_j as $(P_i, 0)$ and $(P_j, 1)$, respectively, that is, these agents move to their destinations along the shortest paths, R_i starts at the moment 0, and R_j starts one step later. Because $\text{dist}_G(s_i, x) \leq \text{dist}_G(s_j, x)$, the routes are not conflicting. In particular, R_j enters each vertex of the common part Q after R_i vacated it. Both R_i and R_j reach the destinations by the moment $\max\{p, q\} + 1 \leq p + q - (\zeta + 2) + 1 = p + q - \zeta - 1$ as the length of P_i and P_j is at least $\zeta + 2$. The remaining agents are routed starting at the moment $\max\{p, q\} + 2 \leq p + q - \zeta$ within the makespan $\sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1$ using Lemma 3. This gives a schedule with total makespan at most

$$\begin{aligned}
 & p + q - \zeta + \sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1 \\
 &= \sum_{h \in [k]} \text{dist}_G(s_h, t_h) + 1 - \zeta \\
 &= \mathcal{L}_{\mathcal{R}, G} - \zeta.
 \end{aligned}$$

XX:12 Routing Multiple Agents Below the Sum of Distances

459 We obtain that $\mathcal{I}$ is a yes-instance. This concludes the proof. $\blacktriangleleft$

460 The next sufficient condition is the existence of two opposite laminar shortest paths of
461 length $\Omega(\zeta)$ such that one of the terminal vertices is sufficiently far from the common part of
462 the paths.

463 **► Lemma 10.** *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT*
464 *MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Suppose that there are distinct $i, j \in [k]$,*
465 *such that*

- 466 (i) $\text{dist}_G(s_i, t_i) \geq \zeta + 2$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 2$,
- 467 (ii) *there are opposite laminar shortest s_i - t_i and s_j - t_j -paths P_i and P_j with $Q = P_i \cap P_j$ being*
468 *an x - y -path such that*
 - 469 $\text{dist}_G(s_i, x) < \text{dist}_G(s_i, y)$ and
 - 470 $\text{dist}_G(s_j, y) < \text{dist}_G(s_j, x)$,
 - 471 $\max\{\text{dist}_G(s_i, x), \text{dist}_G(t_i, y), \text{dist}_G(s_j, y),$
472 $\text{dist}_G(t_j, x)\} \geq \zeta + 2$.

473 *Then, $\mathcal{I}$ is a yes-instance.*

474 **Proof.** By symmetry, assume that $\text{dist}_G(s_i, x) \geq \zeta + 2$; in particular, note that if we exchange
475 the source and the target terminals for each agent then we obtain an equivalent instance.
476 Let P_i and P_j be paths of length p and q , respectively.

477 We consider two cases. First, assume that $\text{dist}_G(s_i, x) > \text{dist}_G(s_j, x)$. We define the routes
478 of R_i and R_j as $(P_i, 0)$ and $(P_j, 0)$. Because $\text{dist}_G(s_i, x) > \text{dist}_G(s_j, x)$, R_i leaves the vertex
479 x before R_j enters this vertex. This implies that the routes of R_i and R_j are not conflicting.
480 These agents reach the destinations by the moment $\max\{p, q\} \leq p + q - (\zeta + 2)$ because
481 $p, q \geq \zeta + 2$. The remaining agents are routed starting at the moment $\max\{p, q\} + 1 \leq p + q - \zeta$.
482 By Lemma 3, they can be routed within the makespan at most $\sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1$.
483 Then we obtain a schedule with the total makespan at most

$$484 \quad p + q - \zeta + \sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1 = \mathcal{L}_{\mathcal{R}, G} - \zeta$$

485 implying that $\mathcal{I}$ is a yes-instance.

486 In the second case, $\text{dist}_G(s_i, x) \leq \text{dist}_G(s_j, x)$. For R_j , we use the route $(P_j, 0)$, that is,
487 this agent is moving from s_j to t_j along P_j starting at zero time. To define the route for R_i ,
488 we set $\ell = \text{dist}_G(s_j, x) - \text{dist}_G(s_i, x) + 1$. Then the route is (P_i, ℓ) . By the choice of ℓ , R_i
489 moves along P_i and reaches x after this vertex was vacated by R_j . Thus, the routes of R_i
490 and R_j are not conflicting. The agent R_j reaches its destination at the moment q , and R_i –
491 at the moment $\ell + 1 + p = \text{dist}_G(s_j, x) - \text{dist}_G(s_i, x) + 1 + p$. The remaining agents are routed
492 starting at the moment $\max\{\ell + 1 + p, q\} + 1$, and for their routing we use a schedule with the
493 makespan at most $\sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1$ using Lemma 3. Notice that $q \leq p + q - \zeta - 1$
494 because $p \geq \zeta + 2$, and $\text{dist}_G(s_j, x) - \text{dist}_G(s_i, x) + 1 + p \leq q - \text{dist}_G(s_i, x) + 1 + p \leq p + q - \zeta - 1$.
495 Then $\max\{\ell + 1 + p, q\} + 1 \leq p + q - \zeta$, and the total makespan is at most

$$496 \quad p + q - \zeta + \sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1 = \mathcal{L}_{\mathcal{R}, G} - \zeta.$$

497 Thus, $\mathcal{I}$ is a yes-instance. This completes the proof. $\blacktriangleleft$

498 In the following lemma, we show that if there exist two pairs of terminals at distances
499 $\Omega(\zeta)$ such that for one of them, there is a shortest path allowing to give way for the second
500 agent, then we have a yes-instance.

501 ► **Lemma 11.** Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT
 502 MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Suppose that there are distinct $i, j \in [k]$
 503 such that
 504 (i) $\text{dist}_G(s_i, t_i) \geq 2\zeta + 4$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 3$,
 505 (ii) there is a shortest s_i - t_i -path P_i containing a vertex w at distance at least $\zeta + 2$ from both
 506 s_i and t_i whose degree in G is at least three.
 507 Then $\mathcal{I}$ is a yes-instance.

508 **Proof.** Let $P_i = (u_0, \dots, u_p)$. By Observation 8, there is a shortest s_j - t_j -path $P_j = v_0, \dots, v_q$
 509 such that P_i and P_j are laminar. If P_i and P_j are codirectional then $\mathcal{I}$ is a yes-instance by
 510 Lemma 9. Assume that the paths are opposite. Let $Q = P_1 \cap P_2$ be an x - y -path and assume
 511 that $\text{dist}_G(s_i, x) < \text{dist}_G(s_i, y)$. If w is not an internal vertex of Q then because $\text{dist}_G(s_i, w) \geq$
 512 $\zeta + 2$ and $\text{dist}_G(t_i, w) \geq \zeta + 2$, we have that $\text{dist}_G(s_i, x) \geq \zeta + 2$ or $\text{dist}_G(t_i, y) \geq \zeta + 2$. Then
 513 $\mathcal{I}$ is a yes-instance by Lemma 10. From now on, we assume that w is an internal vertex of Q .
 514 Since w is an internal vertex of Q and the degree of w is at least three, w has a neighbor
 515 z in G that is not in Q . Taking into account that P_i and P_j are shortest paths, we have
 516 that $z \notin V(P_i)$ and $z \notin V(P_j)$. By symmetry, we assume that $\text{dist}_G(s_j, w) \geq \text{dist}_G(t_j, w)$
 517 as, otherwise, we can replace (s_h, t_h) by (t_h, s_h) for every $h \in [k]$ and obtain an equivalent
 518 instance. We consider two cases.

519 Assume that $\text{dist}_G(s_i, w) \geq \text{dist}_G(s_j, w)$. We define the route of R_i as

$$520 ((u_0, \dots, u_r, z, u_r, \dots, u_p), 0),$$

521 where $u_r = w$. In words, the agent starts moving at the moment 0 and moves along P_i until
 522 it reaches w . Then the agent moves to z , returns to w , and follows P_i up to the destination.
 523 To construct the route for R_j , we set $\ell = \text{dist}_G(s_i, w) - \text{dist}_G(s_j, w) + 1 \geq 0$. Then the route
 524 is defined as (P_j, ℓ) , that is, R_j moves along P_j starting at the moment ℓ . Note that R_i
 525 reaches its destination at the moment $p + 2$, and R_j arrives to t_j at the moment $\ell + q$. By the
 526 definition, R_j reaches w exactly at the same moment when R_i moves to z . Thus, the routes
 527 are not conflicting. As with the proofs of the previous lemmas, the other agents are routed
 528 one step later after the arrival of R_i and R_j , that is, at the moment $\max\{p + 2, \ell + q\} + 1$,
 529 and they are routed within the makespan at most $\sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1$ by making
 530 use of Lemma 3.

531 To upper bound the total makespan, note that since $q \geq \zeta + 3$, $p + 2 \leq p + q - \zeta - 1$.
 532 Because $\text{dist}_G(t_i, w) \geq \zeta + 2$, we have that

$$533 \ell + q = \text{dist}_G(s_i, w) - \text{dist}_G(s_j, w) + 1 + q \leq p - (\zeta + 2) + 1 + q \leq p + q - \zeta - 1.$$

534 Then the makespan is at most

$$535 p + q - \zeta + \sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1 = \mathcal{L}_{\mathcal{R}, G} - \zeta,$$

536 and $\mathcal{I}$ is a yes-instance.

Suppose that $\text{dist}_G(s_i, w) < \text{dist}_G(s_j, w)$. The routes of R_i and R_j are defined sym-
 metrically to the previous case. We set $\ell = \text{dist}_G(s_j, w) - \text{dist}_G(s_i, w) + 1$ and define
 the route of R_i as (P_i, ℓ) . For R_j , we assume that $w = v_r$ and set the route to be
 $((v_0, \dots, v_r, z, v_r, \dots, v_q), 0)$. By the same arguments as in the first case, the routes are not in
 conflict. We have that R_i reaches t_i at the moment $\ell + p$, and R_j arrives to t_j at the moment
 $q + 2$. Then the other agents are routed starting at the moment $\max\{\ell + p, q + 2\} + 1$ within
 the makespan at most $\sum_{h \in [k] \setminus \{i, j\}} \text{dist}_G(s_h, t_h) + 1$ using Lemma 3. As $\text{dist}_G(s_i, w) \geq \zeta + 2$,

$$\ell + p = \text{dist}_G(s_j, w) - \text{dist}_G(s_i, w) + 1 + p \leq q - (\zeta + 2) + 1 + p = p + q - \zeta - 1.$$

XX:14 Routing Multiple Agents Below the Sum of Distances

We also have that $q + 2 \leq p + q - \zeta - 1$ because $p \geq 2\zeta + 4$. This implies that the total makespan is at most $\mathcal{L}_{\mathcal{R},G} - \zeta$ as in the first case. Thus, $\mathcal{I}$ is a yes-instance. This concludes the proof. $\blacktriangleleft$

Lemma 9–Lemma 11 are used to show the following three sufficient conditions that are crucial for our algorithm. In Lemma 12, we show that the existence of three pairs of terminals at distances $\Omega(\zeta)$ is a sufficient condition for having a yes-instance.

► **Lemma 12.** *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R},G} - \lambda$. Suppose that there are distinct $h, i, j \in [k]$, such that*

- (i) $\text{dist}_G(s_h, t_h) \geq 2\zeta + 3$,
 - (ii) $\text{dist}_G(s_i, t_i) \geq \zeta + 2$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 2$.
- Then, $\mathcal{I}$ is a yes-instance.

Proof. Let P_h be a shortest s_h - t_h -path. By Observation 8, there are shortest s_i - t_i and s_j - t_j -paths P_i and P_j , respectively, such that P_h, P_i and P_h, P_j are laminar. If P_h, P_i or P_h, P_j are codirectional then $\mathcal{I}$ is a yes-instance by Lemma 9. Assume that both P_h, P_i and P_h, P_j are opposite. Denote by Q_1 the x_1 - y_1 -path and by Q_2 the x_2 - y_2 -path that are common subpaths of P_h, P_i and P_h, P_j , respectively. We assume that $\text{dist}_G(s_h, x_1) < \text{dist}_G(s_h, y_1)$ and $\text{dist}_G(s_h, x_2) < \text{dist}_G(s_h, y_2)$.

If one of the distances $\text{dist}_G(s_h, x_1)$, $\text{dist}_G(s_h, x_2)$, $\text{dist}_G(t_h, y_1)$, or $\text{dist}_G(t_h, y_2)$ is at least $\zeta + 2$ then $\mathcal{I}$ is a yes-instance by Lemma 10. Suppose that all these distances are at most $\zeta + 1$. Let x be the vertex of P_h at distance $\max\{\text{dist}_G(s_h, x_1), \text{dist}_G(s_h, x_2)\} \leq \zeta + 1$ from s_h , and let y be the vertex at distance $\max\{\text{dist}_G(t_h, y_1), \text{dist}_G(t_h, y_2)\} \leq \zeta + 1$ from t_h . Since $\text{dist}_G(s_h, t_h) \geq 2\zeta + 3$, we have that $\text{dist}_G(s_h, x) < \text{dist}_G(s_h, y)$, that is, $x \neq y$. Note that $Q_1 \cap Q_2$ have the common x - y -subpath Q , and y is closer to s_i and s_j than t_i and t_j , respectively.

Denote by x' and y' the vertices of $V(P_i) \cap V(P_j)$ such that the t_i - x' and s_i - y' -subpaths of P_i do not contain vertices of P_j except x' and y' , respectively. These vertices exists because P_i and P_j are not disjoint. Consider the s_i - t_i -path P'_i obtained by concatenating the s_i - y' -subpath of P_i , the y' - x' -subpath of P_j , and the x' - t_i -subpath of P_i . Since P_i and P_j are shortest path, we have that P'_i and P_j are laminar. Furthermore, because P_i and P_j are shortest paths and s_i is closer to y than to x , we have that Q is a subpath of the y' - x' -subpath of P_j . The paths P'_i and P_j are therefore codirectional. By Lemma 9, we obtain that $\mathcal{I}$ is a yes-instance. This concludes the proof. $\blacktriangleleft$

In Lemma 13, we give sufficient conditions for a yes-instance when there is a pair of terminals at sufficiently big distance.

► **Lemma 13.** *Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R},G} - \lambda$. Let also $I = \{i \in [k] \mid \text{dist}_G(s_i, t_i) \leq \zeta + 1\}$. Suppose that*

- (i) *there is an $h \in [k]$ such that $5\zeta + 4 \leq \text{dist}_G(s_h, t_h) \leq \mathcal{L}_{\mathcal{R},G} - \zeta$,*
 - (ii) $\sum_{i \in I} \text{dist}_G(s_i, t_i) \geq \zeta$.
- Then, $\mathcal{I}$ is a yes-instance. Furthermore, if
- (a) *there is an $h \in [k]$ such that $3\zeta + 2 \leq \text{dist}_G(s_h, t_h)$,*
 - (b) *either $|[k] \setminus I| = 1$ and $\text{dist}_G(s_h, t_h) \leq \mathcal{L}_{\mathcal{R},G} - \zeta$, or $|[k] \setminus I| > 1$ and $1 + \sum_{i \in [k] \setminus I} \text{dist}_G(s_i, t_i) \leq \mathcal{L}_{\mathcal{R},G} - \zeta$, and*
 - (c) $\sum_{i \in I} \text{dist}_G(s_i, t_i) < \zeta$,
- then $\mathcal{I}$ is a yes-instance.

Proof. Both claims are shown by almost the same arguments. Thus, we prove the first claim and then explain how to modify the proof to show the second part of the lemma.

As the claim holds for $\zeta = 0$ by Lemma 3, we assume that $\zeta \geq 1$. We also assume without loss of generality that $5\zeta + 4 \leq \text{dist}_G(s_1, t_1)$. Let $P_1 = v_0, \dots, v_p$ be a shortest s_1 - t_1 -path, and let $P_2, \dots, P_k$ be shortest s_i - t_i -paths for $i \in [2, k]$, respectively.

We select the set of indices $J \subseteq I$ as follows. If there is $i \in I$ such that $\text{dist}_G(s_i, t_i) \geq \zeta$ then we set $J = \{i\}$ for arbitrary i with this property. Otherwise, we greedily choose J be an inclusion minimal set of indices such that $\sum_{i \in J} \text{dist}_G(s_i, t_i) \geq \zeta$. Notice that in this case, $\sum_{i \in J} \text{dist}_G(s_i, t_i) \leq 2\zeta - 2$ since we have that $\text{dist}_G(s_i, t_i) \leq \zeta - 1$ for $i \in I$ when we make the greedy choice.

Then we construct the partition $\{J_1, J_2\}$ of J (either J_1 or J_2 may be empty) where $J_1 = \{i \in J: \text{either } V(P_i) \cap V(P_1) = \emptyset \text{ or there is } j \in [0, 2\zeta + 1] \text{ s.t. } v_j \in V(P_i)\}$ and $J_2 = J \setminus J_1$. The condition that the paths P_i for $i \in J$ are of length at most $\zeta + 1$ implies the following claim.

▷ **Claim 14.** For every $i \in J_1$ and $j \in [3\zeta + 3, p]$, $v_j \notin V(P_i)$, and for every $i \in J_2$ and $j \in [0, 2\zeta + 1]$, $v_j \notin V(P_i)$.

Proof of Claim 14. Notice that for every $i \in [0, 2\zeta + 1]$ and $j \in [3\zeta + 3, p]$, $\text{dist}_G(v_i, v_j) \geq \zeta + 2$ as P_1 is a shortest path. Because for every $h \in [2, k]$, $\text{dist}_G(s_h, t_h) \leq \zeta + 1$, we have that there is no P_h with $h \in [2, k]$ such that P_h and P_1 have common vertices v_i and v_j for all $i \in [0, 2\zeta + 1]$ and $j \in [3\zeta + 3, p]$. This proves the claim ◀

Now we construct a schedule of makespan at most λ .

First, we route the agents R_i with $i \in [k] \setminus J$. Observe that these agents can be routed within the makespan $\lambda = \mathcal{L}_{\mathcal{R}, G} - \zeta$ by Lemma 3 because $\sum_{i \in J} \text{dist}_G(s_i, t_i) \geq \zeta$. Furthermore, they can be routed in such a way that (*) the route of R_1 is along P_1 , and (**) whenever R_1 is in the internal vertices of P_1 , no other vertex is occupied by any other agent. We consider such a schedule for R_i with $i \in [k] \setminus J$, and assume that the route for R_1 is (P_1, ℓ) .

Next, we route the agents R_i with $i \in J$. These agents are routed within the time interval $[\ell + 1, \ell + 5\zeta + 3]$ using the paths P_i for $i \in J$. Thus, because of properties (*) and (**) of the routing of the other agents and the condition that $p \geq 5\zeta + 4$, we only have to ensure that the routes of R_i for $i \in J$ do not conflict with each other and the route of R_1 .

Consider the agents R_i with $i \in J_2$. If $J_2 = \emptyset$ then we do not route. If $|J_2| = 1$ then the unique agent R_i with $i \in J_2$ can be routed along P_i within the makespan $\zeta + 1 \leq 2\zeta$ as $\text{dist}_G(s_i, t_i) \leq \zeta + 1 \leq 2\zeta$. If $|J_2| \geq 2$ then $\sum_{i \in J_2} \text{dist}_G(s_i, t_i) \leq \sum_{i \in I} \text{dist}_G(s_i, t_i) \leq 2\zeta - 2$, and the agents can be routed within the makespan $2\zeta - 1 < 2\zeta$ by Lemma 3 via the paths P_i for $i \in J_2$. In both cases, we can route the agents within the makespan 2ζ using shortest paths avoiding conflicts between the routes. We use this schedule to route the agents with $i \in J_2$ starting at the moment $\ell + 1$. Notice that the agents reach their destination by the moment $2\zeta + 1$. By the choice of J_2 and Claim 14, we have that there are no conflicts between the agents R_i for $i \in J_2$ and R_1 .

The agents R_i with $i \in J_1$ are routed symmetrically using the same arguments. They can be routed within the makespan 2ζ using P_i for $i \in J_1$. We use this schedule to route them starting at the moment $\ell + 3\zeta + 3$. Then they reach their destination by the moment $\ell + 5\zeta + 3$. Therefore, we have that the routes do not conflict with each other and the route of R_1 by Claim 14.

Since the agents R_i with $i \in J_2$ and R_j with $j \in J_1$ are routed within the time intervals $[\ell + 1, \ell + 2\zeta + 1]$ and $[\ell + 3\zeta + 3, \ell + 5\zeta + 3]$, the routes of R_i and R_j are not conflicting. Thus, the constructed schedule is feasible. This proves the first claim.

XX:16 Routing Multiple Agents Below the Sum of Distances

For the second part, we again assume that $\zeta \geq 1$, and because the claim is trivial for $k = 1$, we let $k \geq 2$. We also assume without loss of generality that $3\zeta + 2 \leq \text{dist}_G(s_1, t_1) \leq \mathcal{L}_{\mathcal{R}, G} - \zeta$. Let $P_1 = v_0, \dots, v_p$ be a shortest s_1 - t_1 -path, and let $P_2, \dots, P_k$ be shortest s_i - t_i -paths for $i \in [2, k]$, respectively.

Then we construct the partition $\{J_1, J_2\}$ of I (either J_1 or J_2 may be empty) where $J_1 = \{i \in I : \text{either } V(P_i) \cap V(P_1) = \emptyset \text{ or there is } j \in [0, \zeta + 1] \text{ s.t. } v_j \in V(P_i)\}$ and $J_2 = I \setminus J_1$. In the same way as with Claim 14, we have the following claim about the length of any P_i for $i \in [2, k]$ is at most $\zeta - 1$.

▷ **Claim 15.** For every $i \in J_1$ and $j \in [2\zeta + 1, p]$, $v_j \notin V(P_i)$, and for every $i \in J_2$ and $j \in [0, \zeta + 1]$, $v_j \notin V(P_i)$.

Then we construct a schedule of makespan at most λ .

We route the agents R_i with $i \in [k] \setminus I$. By Lemma 3 and condition (b) of the lemma, there is a routing of makespan at most $\lambda = \mathcal{L}_{\mathcal{R}, G} - \zeta$ such that (*) the route of R_1 goes along P_1 , and (**) whenever R_1 is in the internal vertices of P_1 , no other vertex is occupied by another agent. We assume that the route for R_1 is (P_1, ℓ) .

The agents R_i with $i \in I$ are routed within the time interval $[\ell + 1, \ell + 3\zeta + 1]$ using the paths P_i for $i \in I$. As with the first part of the proof, because of properties (*) and (**) of the routing of the other agents and the condition that $p \geq 5\zeta + 4$, we only have to ensure that the routes of R_i for $i \in I$ do not conflict with each other and the route of R_1 .

First, we deal with the agents R_i with $i \in J_2$. Since $\sum_{i \in J_2} \text{dist}_G(s_i, t_i) \leq \sum_{i \in I} \text{dist}_G(s_i, t_i) \leq \zeta - 1$, the agents can be routed within the makespan ζ by Lemma 3 via the paths P_i for $i \in J_2$. This schedule is used to route the agents with $i \in J_2$ starting at the moment $\ell + 1$. Notice that the agents reach their destination by the moment $\ell + \zeta + 1$. By the choice of J_2 and Claim 15, we have that there are no conflicts between the agents R_i for $i \in J_2$ and R_1 . The agents R_i with $i \in J_1$ can be routed within the makespan ζ using P_i for $i \in J_1$, and we route them starting at the moment $\ell + 2\zeta + 1$. Then they reach their destination by the moment $\ell + 3\zeta + 1$. In the same way as before, the routes do not conflict with each other and the route of R_1 by Claim 15.

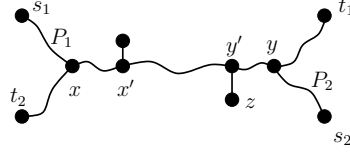
Since the agents R_i with $i \in J_2$ and R_j with $j \in J_1$ are routed within the time intervals $[\ell + 1, \ell + \zeta + 1]$ and $[\ell + 2\zeta + 1, \ell + 3\zeta + 1]$, the routes of R_i and R_j are not conflicting. This completes the proof. ◀

In the following lemma, we provide sufficient conditions for a yes-instance when there are exactly two pairs of terminals at sufficiently big distances.

► **Lemma 16.** Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING, and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. Suppose that there are distinct $i, j \in [k]$ such that

- (i) $\text{dist}_G(s_i, t_i) \geq 5\zeta + 5$ and $\text{dist}_G(s_j, t_j) \geq \zeta + 2$,
 - (ii) the agents R_i and R_j can be routed within the makespan $\mathcal{L}_{\mathcal{R}, G} - \zeta$ without conflicts with each other,
 - (iii) for every $h \in [k]$ such that $h \neq i, j$, $\text{dist}_G(s_h, t_h) \leq \zeta + 1$.
- Then, $\mathcal{I}$ is a yes-instance.

Proof. The claim is trivial for $\zeta = 0$ and it holds for $k = 2$ by (ii). Thus, we assume that $\zeta \geq 1$ and $k \geq 3$. We also assume without loss of generality that $\text{dist}_G(s_1, t_1) \geq 5\zeta + 7$, $\text{dist}_G(s_2, t_2) \geq \zeta + 2$, that is, $i = 1$ and $j = 2$. If $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \geq \zeta$ then $\mathcal{I}$ is a yes-instance by Lemma 13. Therefore, we assume that $\sum_{i=3}^k \text{dist}_G(s_i, t_i) < \zeta$. In particular, $\text{dist}_G(s_i, t_i) \leq \zeta - 1$ for $i \in [3, k]$.



687 ■ **Figure 3** The structure of P_1 and P_2 .

676 By condition (ii), R_1 and R_2 can be routed within the makespan $\mathcal{L}_{\mathcal{R},G} - \zeta$. Denote by r
 677 the minimum makespan for routing R_1 and R_2 . Suppose that $(\mathcal{L}_{\mathcal{R},G} - \zeta) - r \geq \zeta + 1$. Then
 678 we can route R_1 and R_2 using the optimal routing starting at the moment 0. Since they
 679 would reach their destination by the moment r , the other agents can be routed starting at
 680 the moment $r + 1$ using the schedule from Lemma 3. Because $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \leq \zeta - 1$, the
 681 total makespan of the schedule is at most $r + 1 + \zeta \leq \mathcal{L}_{\mathcal{R},G}$ implying that $\mathcal{I}$ is a yes-instance.
 682 By Lemma 3, $r \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) + 1$, and by condition (ii) $r \leq \mathcal{L}_{\mathcal{R},G} - \zeta$. Then
 683 if $r = \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) + 1$, $\mathcal{I}$ is a yes-instance by Lemma 13. From now on, we
 684 assume that these are not the cases, and $\mathcal{L}_{\mathcal{R},G} - 2\zeta \leq r \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2)$.

685 Let $P_1 = (u_0, \dots, u_p)$ and $P_2 = (v_0, \dots, v_q)$ be laminar s_1 - t_1 and s_2 - t_2 -paths, respectively,
 686 such paths exist by Observation 8. For $h \in [3, k]$, denote by P_i a shortest s_i - t_i -path.

687 If P_1 and P_2 are codirectional then $\mathcal{I}$ is a yes-instance by Lemma 9. Thus, we assume that
 688 they are opposite. Let $Q = P_1 \cap P_2$ and denote by x and y its end-vertices assuming that
 689 $\text{dist}_G(s_1, x) < \text{dist}_G(s_1, y)$. If one of the distances $\text{dist}_G(s_1, x)$, $\text{dist}_G(t_2, x)$, $\text{dist}_G(t_1, y)$, and
 690 $\text{dist}_G(s_1, y)$ is at least $\zeta + 2$ then $\mathcal{I}$ is a yes-instance by Lemma 10. Thus, we assume that these
 691 distances are at most $\zeta + 1$. In particular, this means that $|\text{dist}_G(s_1, t_1) - \text{dist}_G(s_2, t_2)| \leq 2\zeta + 2$
 692 and, therefore, $\text{dist}_G(s_2, t_2) \geq 3\zeta + 3$. If there is a vertex w of degree at least three such
 693 that either $w \in V(P_1)$, $\text{dist}_G(s_1, w) \geq \zeta + 2$, and $\text{dist}_G(t_1, w) \geq \zeta + 2$, or $w \in V(P_2)$,
 694 $\text{dist}_G(s_2, w) \geq \zeta + 2$, and $\text{dist}_G(t_2, w) \geq \zeta + 2$, then $\mathcal{I}$ is a yes-instance by Lemma 11.
 695 From now on, we assume that this is not the case. Then Q contains an inclusion maximal
 696 x' - y' -subpath Q' for some $x', y' \in V(Q)$ such that $\text{dist}_G(s_1, x') \leq \zeta + 1$, $\text{dist}_G(t_2, x') \leq \zeta + 1$,
 697 $\text{dist}_G(t_1, y') \leq \zeta + 1$, $\text{dist}_G(s_1, y') \leq \zeta + 1$, and each internal vertex of Q' has degree two in
 698 G . Notice that $\text{dist}_G(x, y) \geq \text{dist}_G(x', y') \geq 3\zeta + 3$. The obtained structure of P_1 and P_2 is
 700 shown in Figure 3.

701 Recall that R_1 and R_2 can be routed with the makespan r . Denote by (W_1, ℓ_1) and
 702 (W_2, ℓ_2) the routes of R_1 and R_2 , respectively, in such a schedule. We consider two cases
 703 depending on whether R_1 and R_2 traverse Q' in this schedule.

704 **Case 1.** For every edge $e \in E(Q')$, both walks W_1 and W_2 traverse e .

705 To deal with this case, we use the following claim.

706 $\triangleright$ **Claim 17.** There is a schedule $\{(W'_1, \ell'_1), (W'_2, \ell'_2)\}$ for R_1 and R_2 with the makespan r
 707 such that either $\ell'_1 = 0$, $W'_1 = P_1$, and $\ell'_2 \geq \text{dist}_G(s_1, y') - \zeta - 2$ or, symmetrically, $\ell'_2 = 0$,
 708 $W'_2 = P_2$, and $\ell'_1 \geq \text{dist}_G(s_2, x') - \zeta - 2$.

709 **Proof of Claim 17.** Consider the walks W_1 and W_2 . Because they traverse all edges of Q' ,
 710 W_1 has a subwalk $A = (a_0, \dots, a_f)$ such that $\{a_0, a_f\} = \{x', y'\}$ and $a_i \in V(Q')$ for $i \in [0, f]$,
 711 and W_2 has a subwalk $B = (b_0, \dots, b_g)$ such that $\{b_0, b_g\} = \{x', y'\}$ and $b_i \in V(Q')$ for
 712 $i \in [0, g]$.

713 For W_1 , we claim that $a_0 = x'$ and $a_f = y'$. For the sake of contradiction, assume
 714 that W_1 first visits y' and then x' . Because x' and y' are vertices of the shortest s_1 - t_1 -
 715 path P_1 , we have that the length of W_1 is at least $\text{dist}_G(s_1, t_1) + 2\text{dist}_G(x', y')$. Because

XX:18 Routing Multiple Agents Below the Sum of Distances

716 $\text{dist}_G(s_2, y'), \text{dist}_G(t_2, x') \leq \zeta + 1$, $\text{dist}_G(s_2, t_2) \leq 2\zeta + 2 + \text{dist}_G(x', y') < 2\text{dist}_G(x', y')$ as
 717 $\text{dist}_G(x', y') \geq 3\zeta + 3$. Thus, the length of W_1 is strictly more than $\text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2)$
 718 contradicting that $r \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2)$. Thus, W_1 traverses Q' from x' to y' .
 719 Symmetrically, we have that $b_0 = y'$ and $b_g = x'$, that is, W_2 goes from y' to x' .

720 Suppose that R_1 appears in x' and start moving along A before or at the same moment
 721 when R_2 appears in y' and begins to follow B . Then because the internal vertices of Q' have
 722 degree two in G , we have that R_2 can appear in y' only when R_1 leaves this vertex. Thus,
 723 R_2 transverse Q' after R_1 . By the definition of Q' , either $t_1 = y'$, or $s_2 = y'$, or y' has a
 724 neighbor z in G such that $z \notin V(P_1)$ (see Figure 3).

725 If $t_1 = y'$, we set $W'_1 = P_1$, $\ell'_1 = 0$, and we define $W'_2 = P_2$ and $\ell'_2 = \text{dist}_G(s_1, y') -$
 726 $\text{dist}_G(s_2, y') + 1 \geq \text{dist}_G(s_1, y') - \zeta$. If $s_1 = y'$, we let $W'_1 = P_1$, $\ell'_1 = 0$, $W'_2 = P_2$, and
 727 $\ell'_2 = \text{dist}_G(s_1, y') + 1 \geq \text{dist}_G(s_1, y')$. In both cases, we have an optimal schedule because
 728 the routes are not conflicting, the agents follow shortest paths, and R_2 visits y' immediately
 729 after R_1 .

730 Suppose that $y' \neq t_1, s_2$. Then y' has a neighbor $z \notin V(P_1)$. Recall that $P_2 = (v_0, \dots, v_q)$
 731 and let $y' = v_h$ for $h \in [q]$. We define $W'_1 = P_1$ and $\ell'_1 = 0$. For R_2 , we set $W'_2 =$
 732 $v_0, \dots, v_h, z, v_h, \dots, v_q$, that is, R_2 follows P_2 up to y' , then goes to z , returns to y' , and
 733 proceeds to t_2 by P_2 , and set $\ell'_2 = \text{dist}_G(s_1, y') - \text{dist}_G(s_2, y') - 1 \geq \text{dist}_G(s_1, y') - \zeta - 2$. As
 734 $z \notin V(P_1)$ and R_2 enters y' after R_1 , the routes do not conflict. Also, because R_1 follows a
 735 shortest path, and R_2 enters y' immediately after this vertex is vacated by R_1 and then R_2
 736 follows a shortest path to t_2 , the schedule is optimal.

737 The case when R_2 appears in y' and start moving along B before R_1 appears in x'
 738 following B is symmetric. By the same arguments as before, we have that we have an optimal
 739 schedule such that $\ell_2 = 0$, $W'_2 = P_2$, and $\ell'_1 \geq \text{dist}_G(s_2, x') - \zeta - 2$. This proves the claim. ◀

740 We use Claim 17 and assume that R_1 uses the route $(P_1 = (u_0, \dots, u_p), 0)$ and R_2 starts its
 741 route (W_2, ℓ_2) at the moment $\ell_2 \geq \text{dist}_G(s_1, y') - \zeta - 2$ (the other case is symmetric). Denote by
 742 U the set of vertices visited by W_2 in the first $\zeta + 2$ steps starting from ℓ_2 . We route the agents
 743 R_i for $i \geq 3$ within the time interval $[3\zeta + 1]$. Notice that $\text{dist}_G(s_1, t_1) \geq \text{dist}_G(x', y') \geq 3\zeta + 3$
 744 and $3\zeta + 1 \leq p$.

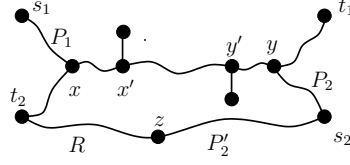
745 We construct the partition $\{J_1, J_2\}$ of $[3, k]$ (either J_1 or J_2 may be empty) where
 746 $J_1 = \{i \in [3, k] : \text{there is } j \in [0, \zeta + 1] \text{ s.t. } u_j \in V(P_i)\}$ and $J_2 = J \setminus J_1$. Then the following
 747 holds.

748 **Claim 18.** For every $i \in J_1$ and $j \in [2\zeta + 1, 3\zeta + 2]$, $u_j \notin V(P_i)$, and for every $i \in J_2$ and
 749 $j \in [0, \zeta + 1]$, $u_j \notin V(P_i)$. Furthermore, for every $i \in J_1$, $V(P_i) \cap U = \emptyset$.

750 **Proof.** The first part holds by the same arguments as Claim 14 because the length of each P_i
 751 for $i \in [3, k]$ is at most $\zeta - 1$. To see the second part, suppose that there is $i \in J_1$ and $w \in U$
 752 such that $w \in V(P_i)$. Since P_i contains a vertex u_j for $j \in [0, \zeta + 1]$ and $\text{dist}_G(s_i, t_i) \leq \zeta - 1$,
 753 we have that $\text{dist}_G(x', u) \leq 2\zeta$. Since $u \in U$, $\text{dist}_G(u, s_2) \leq \zeta + 2$. Then $\text{dist}_G(x', s_2) \leq 3\zeta + 2$.
 754 This contradicts the fact that $\text{dist}_G(x', s_2) \geq \text{dist}_G(x', y') \geq 3\zeta + 3$. This proves the claim. ◀

755 By Lemma 3, the agents R_i with $i \in J_2$ can be routed within the makespan ζ as
 756 $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \leq \zeta - 1$. We route them in the time interval $[1, \zeta + 1]$. By Claim 18, the
 757 paths P_i for $i \in J_2$ have no conflicts with the route of R_1 . Because R_2 does not start its
 758 route until the moment $\ell_2 \geq \text{dist}_G(s_1, y') - \zeta - 2 \geq 2\zeta + 1$, the routes of R_i for $i \in J_2$ has no
 759 conflicts with the route of R_2 .

760 Then we route the agents R_i with $i \in J_1$. Again, they can be routed within the makespan
 761 ζ , and we route them in the time interval $[2\zeta + 1, 3\zeta + 1]$. Note that the routes have no



773 **Figure 4** The construction of P'_2 .

762 conflicts with the routes for R_j with $j \in J_2$. By Claim 18, we also have that the routes have
 763 no conflicts with the route of R_1 . Because $\ell_2 \geq 2\zeta + 1$ and the agents R_i for $i \in J_1$ reach
 764 their destinations by the moment $3\zeta + 1$, any conflict between the routes of R_i and R_2 can
 765 only occur within the first $\zeta + 2$ steps of R_2 starting at the moment ℓ_2 . Thus, R_2 should be in
 766 a vertex of U . However, $V(P_i) \cap U = \emptyset$ by Claim 18. Therefore, there is no conflict between
 767 the routes of R_i for $i \in J_1$ and R_2 .

768 Summarizing, we obtain a schedule of the total makespan at most $r \leq \mathcal{L}_{\mathcal{R},G} - \zeta$. This
 769 concludes the analysis of the first case.

770 **Case 2.** There is an edge $e \in E(Q')$ such that W_1 or W_2 do not traverse e .
 771 Denote by G' the graph obtained from G by the deletion of the internal vertices of Q' . Then
 772 we have the following schedule for R_1 and R_2 .

774 $\triangleright$ **Claim 19.** There is a schedule $\{(P'_1, 0), (P'_2, 0)\}$ for R_1 and R_2 with the makespan r
 775 such that either $P'_1 = P_1$ and P'_2 is an arbitrary shortest s_2 - t_2 -path in G' or, symmetrically,
 776 $P'_2 = P_2$ and P'_1 is an arbitrary shortest s_1 - t_1 -path in G' .

777 **Proof of Claim 19.** Suppose that there is an edge of Q' which is not traversed by W_2 ; the
 778 case when there is an edge of Q' avoided by W_1 is symmetric. Then s_2 and t_2 are in the
 779 same connected component of G' . Let P'_2 be a shortest s_2 - t_2 -path (see Figure 4). Consider
 780 the routes $(P_1, 0)$ and $(P'_2, 0)$ of R_1 and R_2 , respectively. We claim that these routes do
 781 not conflict. By the definition of the routes, a possible conflict may happen only when R_1
 782 goes through the s_1 - x' or y' - s_2 -subpath of P_1 . In the first case, R_1 should be at distance
 783 at most ζ from x' in G' as the conflict cannot happen at the moment 0, and R_1 should
 784 be at distance at most $\zeta + 1$ from y' in the second. Note that when R_1 goes through the
 785 s_1 - x' -subpath of P_1 , R_2 is at distance at most $\text{dist}_G(s_2, y') + \zeta + 1 \leq 2\zeta + 2$ from y' . Since
 786 $\text{dist}_{G'}(x', y') \geq \text{dist}_G(x', y') \geq 3\zeta + 3$, there is no conflict between R_1 and R_2 . For the case
 787 when R_1 goes through the y' - t_1 -subpath of P_1 , note that because $\text{dist}_G(x', y') \geq 3\zeta + 3$, R_2
 788 is at distance at least $3\zeta + 3$ from s_2 in G' . Thus, R_2 is at distance at least $2\zeta + 2$ from y' .
 789 This implies that we have no conflicts between the routes.

790 Clearly, the length of P_1 is at least r , and since W_2 does not traverse Q' , the length of
 791 W_2 is at least the length of P'_2 . Therefore, the constructed schedule is optimal. This proves
 792 the claim. $\blacktriangleleft$

793 By using Claim 19, we first assume that R_1 and R_2 use the routes $(P_1, 0)$ and $(P'_2, 0)$
 794 where P'_2 is a shortest s_2 - t_2 -path in G' . Recall that the minimum makespan $r \geq \mathcal{L}_{\mathcal{R},G} - 2\zeta \geq$
 795 $\text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 2\zeta \geq 6\zeta + 8$, that is, the length of P'_2 is at least $6\zeta + 8$. Thus, r
 796 is the length of P'_2 . Let z be the vertex of P'_2 at distance $3\zeta + 1$ from t_2 in G' , and denote by
 797 $R = (w_0, \dots, w_{3\zeta+1})$ the z - t_2 -subpath of P'_2 (see Figure 4).

798 We define the partition $\{J_1, J_2\}$ of $[3, k]$ (either J_1 or J_2 may be empty) where $J_1 = \{i \in$
 799 $[3, k] : \text{there is } j \in [2\zeta - 1, 3\zeta + 1] \text{ s.t. } w_j \in V(P_i)\}$ and $J_2 = J \setminus J_1$. We have the following
 800 claim.

XX:20 Routing Multiple Agents Below the Sum of Distances

801 $\triangleright$ Claim 20. For every $i \in J_1$ and $j \in [0, \zeta]$, $w_j \notin V(P_i)$, and for every $i \in J_2$ and
 802 $j \in [2\zeta - 1, 3\zeta + 1]$, $w_j \notin V(P_i)$. Furthermore, for every $i \in J_1$ and every $j \in [p - 2\zeta, p]$,
 803 $u_j \notin V(P_i)$.

804 **Proof.** The first claim is fulfilled by the same arguments as Claim 14 as the length of each
 805 P_i for $i \in [3, k]$ is at most $\zeta - 1$. To see the second part, suppose that there is $i \in J_1$ and
 806 $j \in [p - 2\zeta, p]$ such that $u_j \in V(P_i)$. Assume that $w_h \in V(P_1)$ for $h \in J_1$. Notice that w_h is
 807 at distance at most $\zeta + 1$ from t_2 in G , w_h is at distance at most $\zeta - 1$ from u_j . Suppose
 808 that u_j is at distance at most $\zeta + 1$ from y' in G . Then t_2 is at distance at most $3\zeta + 1$ in G .
 809 However, this contradicts the fact that $\text{dist}_G(t_2, y') \geq \text{dist}_G(x', y') \geq 3\zeta + 3$. Thus, u_j is at
 810 distance at least $\zeta + 2$ and at most 2ζ from y' . Then u_h is an internal vertex of Q' as y' is
 811 at distance at most $\zeta + 1$ from t_1 and the length of Q' is at least $3\zeta + 1$. But in this case,
 812 we have that P_i is a subpath of Q' containing only internal vertices of Q' . Then P_i cannot
 813 contain a vertex of P' . This contradiction proves the claim. $\blacktriangleleft$

814 Now we can route the agents R_i for $i \in [3, k]$ within the makespan 3ζ in the time interval
 815 $[r - 3\zeta - 1, r - 1]$.

816 By Lemma 3, the agents R_i for $i \in J_1$ can be routed within the makespan ζ because
 817 $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \leq \zeta - 1$. We route them using this schedule starting at the moment
 818 $r - 3\zeta - 1$. By the first part of Claim 20, there is no conflict between R_2 and R_i for every
 819 $i \in J_1$ because r is the length of P'_2 . Because $r \geq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 2\zeta \geq p + \zeta + 3$,
 820 $r - 3\zeta - 3 \geq p - 2\zeta$. Then by the second part of Claim 20, there is no conflicts between R_i
 821 for $i \in J_1$ and R_1 .

822 Similarly to the agents R_i with $i \in J_1$, the agents with $i \in J_2$ can be routed within the
 823 makespan ζ by Lemma 3. We route them starting at the moment $r - \zeta - 1$. Clearly, these
 824 agents have no conflicts with the agents R_i for $i \in J_1$. Also, they have no conflicts with R_2
 825 by the first part of Claim 20. Finally, because $r \geq p + \zeta + 3$, R_1 is not in G when the agents
 826 R_i with $i \in J_2$ are routed. Therefore, we have no conflicts with R_1 .

827 Summarizing, we obtain a schedule with the makespan $r \leq \mathcal{L}_{\mathcal{R}, G} - \zeta$. Thus, $\mathcal{I}$ is a
 828 yes-instance.

829 The case when R_1 and R_2 use the routes $(P'_1, 0)$ and $(P_2, 0)$ where P'_1 is a shortest
 830 s_1 - t_1 -path in G' is analyzed by the same arguments using symmetry. In fact, this case gets
 831 easier because we have that $r \geq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 2\zeta \geq q + 3\zeta + 5$. Thus, the
 832 agent R_2 following P_2 is going to leave the graph before the moment $r - 3\zeta - 1$. Because
 833 of this, we can route the agents R_i for $i \in [3, k]$ within the time interval $[r - 3\zeta - 1, r - 1]$
 834 avoiding conflicts with R_2 . The remaining arguments are the same as above. This concludes
 835 the proof. $\blacktriangleleft$

836 Now we are ready to prove Theorem 1. We restate it here.

837 **► Theorem 1.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ time*
 838 *on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with $\lambda = \mathcal{L}_{\mathcal{R}, G} - \zeta$.*

839 **Proof.** Let $\mathcal{I} = (G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT
 840 MULTIAGENT PATHFINDING and let $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$. If $\zeta = 0$, then $\mathcal{I}$ is a yes-instance by
 841 Lemma 3. Assume that $\zeta \geq 1$. We also assume that $\text{dist}_G(s_1, t_1) \geq \dots \geq \text{dist}_G(s_k, t_k)$.

842 If $\text{dist}_G(s_1, t_1) > \lambda$, then we have a trivial no-instance. If $k \geq 2$, we use the algorithm
 843 from Lemma 6 to check whether $(G, 2, \{R_1, R_2\}, \lambda)$ is a yes-instance. If $k = 2$, then we solved
 844 the problem. If $k \geq 3$ and we got the no-answer, then $\mathcal{I}$ is a no-instance. Note that the
 845 algorithm runs in polynomial time for two agents. From now on, we assume that $k \geq 3$ and
 846 there is a schedule for R_1 and R_2 with makespan at most λ .

Next, we consider the cases depending on the distances between terminals and either use Lemma 12, Lemma 13, Lemma 16 to argue that $\mathcal{I}$ is a yes-instance or apply Lemma 7 to solve the problem.

Suppose first that $\text{dist}_G(s_1, t_1) \leq 2\zeta + 2$. Then, we have $\sum_{i=1}^k \text{dist}_G(s_i, t_i) \leq (2\zeta + 2)k$ and $\lambda \leq (2\zeta + 2)k$ as $\zeta \geq 1$. In this case, we apply the algorithm from Lemma 7 and solve the problem in $2^{k^2\zeta} \cdot n^{\mathcal{O}(1)}$ time. If $\text{dist}_G(s_1, t_1) \geq 2\zeta + 3$ and $\text{dist}_G(s_2, t_2) \geq \zeta + 2$, then we conclude that $\mathcal{I}$ is a yes-instance by Lemma 12. Thus, from now on, we assume that $\text{dist}_G(s_i, t_i) \leq \zeta + 1$ for each $i \in [3, k]$.

Suppose that $\text{dist}_G(s_1, t_1) \leq 5\zeta + 3$. Then,

$$\lambda \leq \sum_{i=1}^k \text{dist}_G(s_i, t_i) \leq 2(5\zeta + 3) + (k - 2)(\zeta + 1) = k(\zeta + 1) + 8\zeta + 4.$$

We apply the algorithm from Lemma 7 and solve the problem in $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$ time.

From now on, assume that $\text{dist}_G(s_1, t_1) \geq 5\zeta + 4$. If $\text{dist}_G(s_2, t_2) \leq \zeta + 1$, then $\mathcal{I}$ is a yes-instance by Lemma 13: if $\sum_{i=2}^k \text{dist}_G(s_i, t_i) \geq \zeta$ then we apply the first claim and use the second claim otherwise. Furthermore, if $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \geq \zeta$ then $\mathcal{I}$ is a yes-instance even if $\text{dist}_G(s_2, t_2) \geq \zeta + 2$ by the same lemma.

Now, we may assume $\text{dist}_G(s_2, t_2) \geq \zeta + 2$ and $\sum_{i=3}^k \text{dist}_G(s_i, t_i) \leq \zeta - 1$. If $\text{dist}_G(s_1, t_1) \geq 5\zeta + 5$, then $\mathcal{I}$ is a yes-instance by Lemma 16. Hence, it remains to consider the case when $\text{dist}_G(s_1, t_1) \leq 5\zeta + 4$. Here, we have that $\lambda \leq \sum_{i=1}^k \text{dist}_G(s_i, t_i) \leq 2(5\zeta + 4) + \zeta - 1 = 11\zeta + 7$ and the problem can be solved in $2^{\mathcal{O}(k\zeta)} \cdot n^{\mathcal{O}(1)}$ time by Lemma 7. This concludes the case analysis.

Since the distances between terminals can be computed in polynomial time by standard algorithms (see, e.g., [7]), the overall running time is $2^{\mathcal{O}(k^2\zeta)} \cdot n^{\mathcal{O}(1)}$, concluding the proof. ◀

5 The case of distinct terminals

In this section, we prove that TRANSIENT MULTIAGENT PATHFINDING is FPT when parameterized by only $\zeta = \mathcal{L}_{\mathcal{R}, G} - \lambda$ if the terminals are pairwise distinct. The result is based on the following lemma.

► **Lemma 21.** *Let G be a connected graph and let $R_1 = (s_1, t_1)$, $R_2 = (s_2, t_2)$, and $R_3 = (s_3, t_3)$ be agents such that the terminal vertices $s_1, s_2, s_3, t_1, t_2, t_3$ are distinct. Then there are distinct $i, j \in [3]$ such that the agents R_i and R_j can be routed with the makespan at most $\text{dist}_G(s_i, t_i) + \text{dist}_G(s_j, t_j) - 1$ without conflicts.*

Proof. Assume that $\text{dist}_G(s_1, t_1) \geq \text{dist}_G(s_2, t_2)$, $\text{dist}_G(s_3, t_3)$, and let $P_1 = (u_0, \dots, u_p)$ be a shortest s_1 - t_1 -path. We select shortest s_2 - t_2 and s_3 - t_3 -paths P_2 and P_3 , respectively, in such a way that P_1, P_2 and P_1, P_3 are laminar using Observation 8.

Suppose that there is $i \in [2, 3]$ such that P_1 and P_i are codirectional. We assume without loss of generality that $i = 2$. If P_1 and P_2 have no common vertices then the routes $(P_1, 0)$ and $(P_2, 0)$ are not conflicting, and we have a routing with the makespan $\text{dist}_G(s_1, t_1) \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 1$. Assume that $V(P_1) \cap V(P_2) \neq \emptyset$. Note that $\text{dist}_G(s_1, t_1) \geq 2$ in this case. Because P_1 and P_2 are laminar, their intersection is an x - y -path Q that is a common subpath of P_1 and P_2 . We assume that $\text{dist}_G(s_1, x) \leq \text{dist}_G(s_1, y)$.

Suppose that $\text{dist}_G(s_1, x) \neq \text{dist}_G(s_2, x)$. Then the routes $(P_1, 0)$ and $(P_2, 0)$ are not conflicting because the agent whose s -terminal is closer to x reaches x first and then follows the route. Thus, we can assume that $\text{dist}_G(s_1, x) = \text{dist}_G(s_2, x)$. Because $s_1 \neq s_2$, we can route R_1 and R_2 using $(P_1, 0)$ and $(P_2, 1)$, respectively. Then because R_2 reaches x after R_1 ,

XX:22 Routing Multiple Agents Below the Sum of Distances

there are no conflicts. As the terminals are distinct and $\text{dist}_G(s_1, t_1) \geq \text{dist}_G(s_2, t_2)$, we have that $\text{dist}_G(s_1, t_1) \geq 2$. Then the total makespan is $\max\{\text{dist}_G(s_1, t_1), \text{dist}_G(s_2, t_2) + 1\}$. If $\text{dist}_G(s_1, t_1) > \text{dist}_G(s_2, t_2)$ then the makespan is $\text{dist}_G(s_1, t_1) \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 1$. If $\text{dist}_G(s_1, t_1) = \text{dist}_G(s_2, t_2)$ then the makespan is $\text{dist}_G(s_2, t_2) + 1 \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 1$ as $\text{dist}_G(s_1, t_1) \geq 2$. In both cases, the makespan is upper bounded by $\text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 1$. Thus, the claim holds if P_1 and P_2 are codirectional.

Suppose now that P_1, P_2 and P_1, P_3 are opposite. Consider P_1 and P_2 , and let an x - y -path Q be the common subpath of P_1 and P_2 where $\text{dist}_G(s_1, x) < \text{dist}_G(s_1, y)$.

Assume that $x \notin \{s_1, t_2\}$. If $\text{dist}_G(s_1, x) > \text{dist}_G(s_2, x)$ then the routes $(P_1, 0)$ and $(P_2, 0)$ are not conflicting, and the makespan for this schedule is $\text{dist}_G(s_1, t_1) \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 1$. Let $\text{dist}_G(s_1, x) \leq \text{dist}_G(s_2, x)$. Consider the routes $(P_1, \text{dist}_G(s_2, x) - \text{dist}_G(s_1, x) + 1)$ and $(P_2, 0)$ of R_1 and R_2 , respectively. Because R_1 enters x after this vertex is vacated by R_2 , the routes are not conflicting. Since $x \notin \{s_1, t_2\}$, $\text{dist}_G(s_2, x) - \text{dist}_G(s_1, x) + 1 \leq \text{dist}_G(s_2, t_2) - 1$. As $\text{dist}_G(s_1, t_1) \geq \text{dist}_G(s_2, t_2)$, the makespan of the schedule is $\text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, x) - \text{dist}_G(s_1, x) + 1 \leq \text{dist}_G(s_1, t_1) + \text{dist}_G(s_2, t_2) - 1$.

By symmetry, it remains to consider the case when $x \in \{s_1, t_2\}$ and $y \in \{s_2, t_1\}$. Because the terminals are distinct and $\text{dist}_G(s_1, t_1) \geq \text{dist}_G(s_2, t_2)$, we have that either $P_2 = (u_{p-1}, \dots, u_1)$, or $P_2 = (s_2, u_p, \dots, u_1)$, or $P_2 = (u_{p-1}, \dots, u_0, t_2)$. By the same arguments, either $P_3 = (u_{p-1}, \dots, u_1)$, or $P_3 = (s_3, u_p, \dots, u_1)$, or $P_3 = (u_{p-1}, \dots, u_0, t_3)$. Then, P_2 and P_3 are laminar codirectional paths. By the same arguments as in the case when P_1 is codirectional with P_2 or P_3 , we have that R_2 and R_3 can be routed with the makespan $\text{dist}_G(s_2, t_2) + \text{dist}_G(s_3, t_3) - 1$. This concludes the proof. $\blacktriangleleft$

By Lemma 21, we can show the following.

► **Lemma 22.** *Let $(G, k, \mathcal{R}, \lambda)$ be an instance of TRANSIENT MULTIAGENT PATHFINDING such that the terminals are pairwise distinct. If $\lambda \geq \sum_{i \in [k]} \text{dist}(s_i, t_i) - \lfloor \frac{k-1}{2} \rfloor$, then $(G, k, \mathcal{R}, \lambda)$ is a yes-instance.*

Proof. If $k = 1$ then the claim is trivial. If $k = 2$ then we route R_1 and R_2 using shortest s_1 - t_1 - and s_2 - t_2 -paths P_1 and P_2 , respectively, using the routes $(P_1, 0)$, $(P_2, \text{dist}_G(s_1, t_1))$. Since $t_1 \neq s_2$, there are no conflicts and this proves the claim. Assume that $k \geq 3$.

We greedily select $r = \lfloor \frac{k-1}{2} \rfloor$ pairs $\{i_1, j_1\}, \dots, \{i_r, j_r\}$ of indices such that all the indices are distinct and for every $h \in [r]$, the agents R_{i_h} and R_{j_h} can be routed with the makespan at most $\text{dist}_G(s_{i_h}, t_{i_h}) + \text{dist}_G(s_{j_h}, t_{j_h}) - 1$ without conflicts. This is possible due to Lemma 21. We fix these routes and concatenate them, that is R_{i_1} and R_{j_1} start at the moment 0, and for each $h \in [2, r]$, R_{i_h} and R_{j_h} are routed when $R_{i_{h-1}}$ and $R_{j_{h-1}}$ reach their destinations. Then, we consecutively append the routes for $h \in [k] \setminus (\{i_1, \dots, i_r\} \cup \{j_1, \dots, j_r\})$ using shortest s_h - t_h -paths. Because the terminals are distinct, we obtain a feasible schedule with the total makespan at most $\sum_{i \in [k]} \text{dist}(s_i, t_i) - \lfloor \frac{k-1}{2} \rfloor$. This proves the lemma. $\blacktriangleleft$

We remark that Lemma 22 implies that if the terminals are distinct then $(G, k, \mathcal{R}, \lambda)$ is a yes-instance of TRANSIENT MULTIAGENT PATHFINDING if $\lambda \leq \mathcal{L}_{\mathcal{R}, G}^* := \sum_{i \in [k]} \text{dist}(s_i, t_i)$.

Combining Theorem 1 and Lemma 22, we obtain the algorithmic result about TRANSIENT MULTIAGENT PATHFINDING parameterized below $\mathcal{L}_{\mathcal{R}, G}^*$ for distinct terminals.

► **Theorem 2.** *TRANSIENT MULTIAGENT PATHFINDING can be solved in $2^{\mathcal{O}(\zeta^3)} \cdot n^{\mathcal{O}(1)}$ time on instances $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ with pairwise distinct terminals and $\lambda = \mathcal{L}_{\mathcal{R}, G}^* - \zeta$.*

934 **Proof.** Let $(G, k, \mathcal{R} = \{R_i = (s_i, t_i) \mid i \in [k]\}, \lambda)$ be an instance of TRANSIENT MULTIAGENT
 935 PATHFINDING with pairwise distinct terminals and $\lambda = \mathcal{L}_{\mathcal{R}, G}^* - \zeta$. If $\zeta \leq \lfloor \frac{k-1}{2} \rfloor$, this is
 936 a yes-instance by Lemma 22. Otherwise, $k \leq 2\zeta$ and we can apply the algorithm from
 937 Theorem 1 and solve the problem in $2^{\mathcal{O}(\zeta^3)} \cdot n^{\mathcal{O}(1)}$ time. ◀

938 6 Computational lower bounds

939 In this section, we prove our hardness results. For this, it will be convenient to consider
 940 LAYERED VERTEX-DISJOINT SHORTEST PATHS [5]. A graph is layered if there is a partition
 941 of the vertices into (disjoint) sets $V_1, V_2, \dots, V_\lambda$ such that all edges $\{u, v\}$ are between two
 942 consecutive layers, that is $u \in V_i$ and $v \in V_{i+1}$ or $u \in V_{i+1}$ and $v \in V_i$ for some $i \in [\lambda - 1]$.
 943 An instance of LAYERED VERTEX-DISJOINT SHORTEST PATHS consists of a layered graph
 944 and k terminal pairs (s_i, t_i) such that $s_i \in V_1$, $t_i \in V_\lambda$, and each shortest s_i - t_i -path contains
 945 exactly one vertex of each layer. The question is whether there are k shortest paths (one
 946 between each terminal pair) that are pairwise vertex-disjoint. It is known that LAYERED
 947 VERTEX-DISJOINT SHORTEST PATHS does not admit a polynomial kernel parameterized
 948 by $k + \lambda$ (the number of terminal pairs plus the number of layers) [5]. Moreover, parameterized
 949 by only k , the problem is known to be W[1]-hard [30]. We present a reduction from LAYERED
 950 VERTEX-DISJOINT SHORTEST PATHS to TRANSIENT MULTIAGENT PATHFINDING based on
 951 a reduction by [20, 21].

952 ▶ **Theorem 23.** *TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard parameterized by k .*

953 **Proof.** We reduce from LAYERED VERTEX-DISJOINT SHORTEST PATHS parameterized by
 954 the number k' of terminal pairs, which is known to be W[1]-hard [30]. Let λ' be the number
 955 of layers in the input graph. For each terminal pair (s_i, t_i) , we construct an agent $R_i = (s_i, t_i)$
 956 and we set $\lambda = \lambda' - 1$. We next show that the reduction is correct. First assume that
 957 there is a set of k vertex-disjoint shortest paths. Then, we let each agent appear in time
 958 step 0, and let them move along the shortest path until reaching t_i in time step $\lambda' - 1 = \lambda$.
 959 Since all paths are vertex-disjoint, this shows that the constructed instance is a yes-instance.
 960 In the other direction, assume that there is a schedule for all agents. Since the distance
 961 between s_i and t_i for each agent is by assumption $\lambda' - 1 = \lambda$, the schedule has to let all
 962 agents appear at their respective starting vertices at time step 0 and let them move along
 963 their respective shortest paths towards their destinations. Hence, each agent is in layer i
 964 at time step $i - 1$ in the assumed schedule. This shows that the paths taken by all agents
 965 are pairwise vertex-disjoint and thus the original instance of LAYERED VERTEX-DISJOINT
 966 SHORTEST PATHS is a yes-instance. ◀

967 We mention in passing that the above result can also be modified to show W[1]-hardness
 968 with respect to k even if the input graph is subcubic as shown by [21]. Simply replace each
 969 vertex by an in-tree followed by an out-tree (both of height $\log(n)$) and replacing each edge
 970 by an edge between a unique out-leaf of the vertex in the earlier layer and a unique in-leaf of
 971 the vertex in the latter layer.

972 ▶ **Corollary 24.** *TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard when parameterized
 973 by k even if the input graph is subcubic.*

974 We next show W[1]-hardness parameterized by ζ , the difference between the upper bound
 975 on the makespan and λ .

976 ▶ **Theorem 25.** *TRANSIENT MULTIAGENT PATHFINDING is W[1]-hard parameterized by ζ .*

Proof. We reduce from TRANSIENT MULTIAGENT PATHFINDING parameterized by k , which is $W[1]$ -hard by Theorem 23. Let $(G, k, \mathcal{R}, \lambda)$ be an input instance. We construct an equivalent instance $(G', k', \mathcal{R}', \lambda')$ with $k' = k + \lambda$ and $\lambda' = \lambda$ as follows. First, we add to G a set of λ new vertices $u_1, u_2, \dots, u_\lambda$. Next, we add a new vertex v and make it adjacent to all other vertices in G (including the previously created vertices). We set $\mathcal{R}' = \mathcal{R} \cup \{(v, u_i) \mid i \in [\lambda]\}$. Note that v ensures that the distance between each terminal pair in $\mathcal{R}$ (in G') is at most 2 and the distance between all other terminal pairs is 1. Hence, ζ is at most $1 + 2k + \lambda - \lambda = 1 + 2k$. Since the reduction can clearly be computed in polynomial time, it only remains to show correctness.

To this end, first assume that the original instance is a yes-instance. Then, we follow the same schedule for each agent in $\mathcal{R}$. For each new agent (v, u_i) , we let the agent appear at vertex v at time step $i - 1$ and move to vertex u_i at time step i . Note that each agent reaches its destination latest at time step λ and no vertex is occupied by more than one agent at any time. Thus, the constructed instance is a yes-instance.

In the other direction, assume that the constructed instance is a yes-instance. Then, each new agent (v, u_i) has to appear at vertex v at a distinct time step. Let (v, u_j) be the last agent appearing. Since u_j has to be reached by time step λ , it holds that u_j appears at v at time step at most $\lambda - 1$. Hence, at each time step in $[0, \lambda - 1]$ a new agent appears at vertex v . Thus, no agent in $\mathcal{R}$ can occupy vertex v before time step λ . Since $v \neq t_i$ for each agent, no agent in $\mathcal{R}$ occupies v at any time step. Thus, all agents follow a schedule in the original graph G and reach their destination by time step λ . That is, the original instance of TRANSIENT MULTIAGENT PATHFINDING is a yes-instance, concluding the proof. ◀

We next prove Theorem 26, that it is unlikely that TRANSIENT MULTIAGENT PATHFINDING has a polynomial kernel parameterized by $k + \zeta$. This result is shown combining ideas behind Theorems 23 and 25.

► **Theorem 26.** *TRANSIENT MULTIAGENT PATHFINDING parameterized by $k + \zeta$ does not admit a polynomial kernel unless $\text{coNP} \subseteq \text{NP}/\text{poly}$.*

Proof. We design a polynomial parameter transformation from LAYERED VERTEX-DISJOINT SHORTEST PATHS parameterized by $k + \lambda$. This problem is known to not admit a polynomial kernel [5]. Let $(G, \{(s_i, t_i) \mid i \in [k]\})$ be an input instance of LAYERED VERTEX-DISJOINT SHORTEST PATHS and let λ be the number of layers in G . We construct an equivalent instance $(G', k', \mathcal{R}', \lambda')$ with $k' = k + \lambda - 1$ and $\lambda' = \lambda - 1$ as follows. First, we add to G a set of $\lambda - 1$ new vertices $u_1, u_2, \dots, u_{\lambda-1}$. Next, we add a new vertex v and make it adjacent to all other vertices in G (including the previously created vertices). We set $\mathcal{R}' = \{(s_i, t_i) \mid i \in [k]\} \cup \{(v, u_i) \mid i \in [\lambda - 1]\}$. Note that v ensures that the distance between each terminal pair (s_i, t_i) is at most 2 (in G') and the distance between all other terminal pairs is 1 by construction. Hence, ζ is at most $1 + 2k + \lambda - 1 - (\lambda - 1) = 1 + 2k$. Since the polynomial parameter transformation can clearly be computed in polynomial time, it only remains to show correctness.

To this end, first assume that the original instance is a yes-instance. Let P_i be a path from s_i to t_i of length $\lambda - 1$ for each $i \in [k]$. For each agent (s_i, t_i) , we let the agent appear at s_i at time step 0 and then follow P_i so it reaches t_i at time step $\lambda - 1 = \lambda'$. For each agent (v, u_i) , we let them appear at v at time step $i - 1$ and move to u_i at time step i . Note that each agent reaches its destination latest at time step λ' and no vertex is occupied by more than one agent at any time. Thus, the constructed instance is a yes-instance.

In the other direction, assume that the constructed instance is a yes-instance. Then, each agent (v, u_i) has to appear at vertex v at a distinct time step. Let (v, u_j) be the last agent

appearing. Since u_j has to be reached by time step $\lambda' = \lambda - 1$, it holds that u_j appears at v at time step at most $\lambda - 2$. Hence, at each time step in $[0, \lambda - 2]$ a new agent appears at vertex v . Thus, no agent (s_i, t_i) can occupy vertex v before time step $\lambda - 1$. Since $v \neq t_i$ for each agent, no agent in (s_i, t_i) occupies v at any time step. Thus, all these agents follow a schedule in the original graph G and reach their destination by time step $\lambda' = \lambda - 1$. Note that this implies that each such agent occupies a vertex of layer i at time step $i - 1$ for each $i \in [\lambda]$. Thus, all these paths are vertex-disjoint showing that the original instance of LAYERED VERTEX-DISJOINT SHORTEST PATHS is a yes-instance. ◀

7 Conclusion and open questions

In this work, we investigated TRANSIENT MULTIAGENT PATHFINDING through the lens of the above-and-below-guarantee paradigm of parameterized complexity. Our results provide a near-complete understanding of the problem with respect to the parameter k and ζ . We established fixed-parameter tractability for the combined parameterization and complement this with matching lower bounds showing that neither k nor ζ alone is likely to yield an FPT algorithm. In addition, we ruled out the existence of a polynomial kernel for the combined parameter. Together, these results delineate the boundary between tractability and hardness for the problem in a rather precise manner.

We conclude with several open questions that we find particularly interesting.

- **Alternative upper bounds.** From a combinatorial perspective, we were unable to prove or disprove whether the following inequality

$$\lambda \leq 2 \cdot \max_{i \in [k]} \text{dist}(s_i, t_i) + k$$

holds for the minimum makespan λ in general. Establishing such a bound (or providing a counterexample) would be of independent interest. A positive answer would, of course, give rise to new algorithmic questions within the above-and-below-guarantee framework.

- **Parameterization by ζ .** The precise complexity of TRANSIENT MULTIAGENT PATHFINDING when parameterized solely by ζ remains unresolved. Is the problem in XP or para-NP-hard? Clarifying this would complete the single-parameter landscape.
- **Restricted graph classes.** Another promising direction is to study the complexity of the problem on restricted graph classes, such as planar graphs or graphs of bounded degree. In particular, it remains open whether the problem is FPT when parameterized by k on planar graphs or even on planar grids.

References

- 1 Noga Alon, Gregory Gutin, Eun Jung Kim, Stefan Szeider, and Anders Yeo. Solving MAX- r -SAT above a tight lower bound. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 511–517. SIAM, 2010.
- 2 Noga Alon, Raphael Yuster, and Uri Zwick. Color-coding. *J. ACM*, 42(4):844–856, 1995. URL: <http://dx.doi.org/10.1145/210332.210337>, doi:10.1145/210332.210337.
- 3 Jacopo Banfi, Nicola Basilico, and Francesco Amigoni. Intractability of time-optimal multirobot path planning on 2D grid graphs with holes. *IEEE Robotics and Automation Letters*, 2(4):1941–1947, 2017.
- 4 Bahareh Banyassady, Mark de Berg, Karl Bringmann, Kevin Buchin, Henning Fernau, Dan Halperin, Irina Kostitsyna, Yoshio Okamoto, and Stijn Slot. Unlabeled multi-robot motion planning with tighter separation bounds. In *SoCG*, volume 224 of *LIPIcs*, pages 12:1–12:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

- 1068 5 Matthias Bentert, Fedor V. Fomin, and Petr A. Golovach. Tight approximation and kernel-
1069 ization bounds for vertex-disjoint shortest paths. In *Proceedings of the 42nd International*
1070 *Symposium on Theoretical Aspects of Computer Science (STACS)*, pages 17:1–17:17. Schloss
1071 Dagstuhl - Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPICS.STACS.2025.17.
- 1072 6 Ivona Bezáková, Radu Curticapean, Holger Dell, and Fedor V. Fomin. Finding detours is
1073 fixed-parameter tractable. *SIAM J. Discrete Math.*, 33(4):2326–2345, 2019. doi:10.1137/
1074 17M1148566.
- 1075 7 Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction*
1076 *to Algorithms, 3rd Edition*. MIT Press, 2009. URL: [http://mitpress.mit.edu/books/](http://mitpress.mit.edu/books/introduction-algorithms)
1077 [introduction-algorithms](http://mitpress.mit.edu/books/introduction-algorithms).
- 1078 8 Robert Crowston, Mark Jones, Gabriele Muciaccia, Geevarghese Philip, Ashutosh Rai, and
1079 Saket Saurabh. Polynomial kernels for lambda-extendible properties parameterized above the
1080 Poljak-Turzík bound. In *IARCS Annual Conference on Foundations of Software Technology*
1081 *and Theoretical Computer Science (FSTTCS)*, volume 24 of *Leibniz International Proceedings*
1082 *in Informatics (LIPIcs)*, pages 43–54, Dagstuhl, Germany, 2013. Schloss Dagstuhl–Leibniz-
1083 Zentrum fuer Informatik.
- 1084 9 Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin
1085 Pilipczuk, Michal Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015.
1086 doi:10.1007/978-3-319-21275-3.
- 1087 10 Argyrios Deligkas, Eduard Eiben, Robert Ganian, Iyad Kanj, Dominik Leko, and M. S.
1088 Ramanujan. Routing few robots in a crowded network. *J. Comput. Syst. Sci.*, 157:103753, 2026.
1089 URL: <https://doi.org/10.1016/j.jcss.2025.103753>, doi:10.1016/J.JCSS.2025.103753.
- 1090 11 Argyrios Deligkas, Eduard Eiben, Robert Ganian, Iyad Kanj, and M. S. Ramanujan. Parame-
1091 terized algorithms for coordinated motion planning: Minimizing energy. In Karl Bringmann,
1092 Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on*
1093 *Automata, Languages, and Programming, ICALP 2024, Tallinn, Estonia, July 8-12, 2024*,
1094 volume 297 of *LIPIcs*, pages 53:1–53:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik,
1095 2024. Full version available at <https://arxiv.org/abs/2404.15950>.
- 1096 12 Argyrios Deligkas, Eduard Eiben, Robert Ganian, Iyad Kanj, and M. S. Ramanujan. Parameter-
1097 ized algorithms for multiagent pathfinding on trees. In Sanmay Das, Ann Nowé, and Yevgeniy
1098 Vorobeychik, editors, *Proceedings of the 24th International Conference on Autonomous Agents*
1099 *and Multiagent Systems, AAMAS 2025, Detroit, MI, USA, May 19-23, 2025*, pages 584–592.
1100 International Foundation for Autonomous Agents and Multiagent Systems / ACM, 2025. URL:
1101 <https://dl.acm.org/doi/10.5555/3709347.3743574>, doi:10.5555/3709347.3743574.
- 1102 13 Erik D. Demaine, Sándor P. Fekete, Phillip Keldenich, Henk Meijer, and Christian Scheffer.
1103 Coordinated motion planning: Reconfiguring a swarm of labeled robots with bounded stretch.
1104 *SIAM Journal on Computing*, 48(6):1727–1762, 2019.
- 1105 14 Erik D. Demaine, Mohammad Taghi Hajiaghayi, and Dániel Marx. Minimizing movement:
1106 Fixed-parameter tractability. *ACM Trans. Algorithms*, 11(2):14:1–14:29, 2014. doi:10.1145/
1107 2650247.
- 1108 15 Erik D. Demaine and Mikhail Rudoy. A simple proof that the $(n^2 - 1)$ -puzzle is hard.
1109 *Theoretical Computer Science*, 732:80–84, 2018.
- 1110 16 Reinhard Diestel. *Graph theory*, volume 173 of *Graduate Texts in Mathematics*. Springer-Verlag,
1111 Berlin, 5th edition, 2017.
- 1112 17 Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*.
1113 Texts in Computer Science. Springer, 2013.
- 1114 18 Eduard Eiben, Robert Ganian, and Iyad Kanj. The parameterized complexity of coordinated
1115 motion planning. In Erin W. Chambers and Joachim Gudmundsson, editors, *39th International*
1116 *Symposium on Computational Geometry, SoCG 2023, June 12-15, 2023, Dallas, Texas, USA*,
1117 volume 258 of *LIPIcs*, pages 28:1–28:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik,
1118 2023. URL: <https://doi.org/10.4230/LIPICS.SoCG.2023.28>, doi:10.4230/LIPICS.SoCG.
1119 2023.28.

- 1120 19 Eduard Eiben, Robert Ganian, Iyad Kanj, and M. S. Ramanujan. A minor-testing approach
1121 for coordinated motion planning with sliding robots. In Oswin Aichholzer and Haitao Wang,
1122 editors, *41st International Symposium on Computational Geometry, SoCG 2025, Kanazawa,
1123 Japan, June 23-27, 2025*, volume 332 of *LIPIcs*, pages 44:1–44:15. Schloss Dagstuhl - Leibniz-
1124 Zentrum für Informatik, 2025. URL: <https://doi.org/10.4230/LIPIcs.SocG.2025.44>, doi:
1125 10.4230/LIPIcs.SocG.2025.44.
- 1126 20 Foivos Fioravantes, Dušan Knop, Jan Matyáš Křišťan, Nikolaos Melissinos, and Michal Opler.
1127 Exact algorithms and lowerbounds for multiagent path finding: Power of treelike topology.
1128 In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth
1129 AAAI Conference on Artificial Intelligence*, pages 17380–17388. AAAI Press, 2024. URL:
1130 <https://doi.org/10.1609/aaai.v38i16.29686>.
- 1131 21 Foivos Fioravantes, Dusan Knop, Jan Matyáš Kristan, Nikolaos Melissinos, and Michal Opler.
1132 Exact algorithms for multiagent path finding with communication constraints on tree-like
1133 structures. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI)*,
1134 pages 23177–23185. AAAI Press, 2025. doi:10.1609/AAAI.V39I22.34483.
- 1135 22 Fedor V. Fomin, Petr A. Golovach, Daniel Lokshtanov, Fahad Panolan, Saket Saurabh, and
1136 Meirav Zehavi. Multiplicative parameterization above a guarantee. *ACM Trans. Comput.
1137 Theory*, 13(3):18:1–18:16, 2021. doi:10.1145/3460956.
- 1138 23 Steven Fortune, John E. Hopcroft, and James Wyllie. The directed subgraph homeomorphism
1139 problem. *Theor. Comput. Sci.*, 10:111–121, 1980. doi:10.1016/0304-3975(80)90009-2.
- 1140 24 Shivam Garg and Geevarghese Philip. Raising the bar for vertex cover: Fixed-parameter
1141 tractability above a higher guarantee. In *Proceedings of the Twenty-Seventh Annual ACM-
1142 SIAM Symposium on Discrete Algorithms (SODA)*, pages 1152–1166. SIAM, 2016. doi:
1143 10.1137/1.9781611974331.ch80.
- 1144 25 Tzvika Geft and Dan Halperin. Refined hardness of distance-optimal multi-agent path finding.
1145 In *AAMAS*, pages 481–488, 2022.
- 1146 26 Gregory Z. Gutin and Matthias Mnich. A survey on graph problems parameterized above and
1147 below guaranteed values. *CoRR*, abs/2207.12278, 2022. [arXiv:2207.12278](https://arxiv.org/abs/2207.12278), doi:10.48550/
1148 [arXiv.2207.12278](https://arxiv.org/abs/2207.12278).
- 1149 27 Gregory Z. Gutin, Arash Rafiey, Stefan Szeider, and Anders Yeo. The linear arrangement
1150 problem parameterized above guaranteed value. *Theory Comput. Syst.*, 41(3):521–538, 2007.
1151 doi:10.1007/s00224-007-1330-6.
- 1152 28 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic
1153 determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*,
1154 4(2):100–107, 1968.
- 1155 29 Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven
1156 Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Thirty-Fifth AAAI
1157 Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative
1158 Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational
1159 Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pages
1160 11272–11281. AAAI Press, 2021. URL: <https://doi.org/10.1609/aaai.v35i13.17344>, doi:
1161 10.1609/AAAI.V35I13.17344.
- 1162 30 William Lochet. A polynomial time algorithm for the k -disjoint shortest paths problem.
1163 In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages
1164 169–178. SIAM, 2021. doi:10.1137/1.9781611976465.12.
- 1165 31 Daniel Lokshtanov, N. S. Narayanaswamy, Venkatesh Raman, M. S. Ramanujan, and Saket
1166 Saurabh. Faster parameterized algorithms using linear programming. *ACM Trans. Algorithms*,
1167 11(2):15:1–15:31, 2014. doi:10.1145/2566616.
- 1168 32 Meena Mahajan, Venkatesh Raman, and Somnath Sikdar. Parameterizing above or below
1169 guaranteed values. *J. Computer and System Sciences*, 75(2):137–153, 2009.
- 1170 33 Robert Morris, Corina S. Pasareanu, Kasper Søe Luckow, Waqar Malik, Hang Ma, T. K. Satish
1171 Kumar, and Sven Koenig. Planning, scheduling and monitoring for airport surface operations.

- 1172 In Daniele Magazzeni, Scott Sanner, and Sylvie Thiébaux, editors, *Planning for Hybrid*
1173 *Systems, Papers from the 2016 AAAI Workshop, Phoenix, Arizona, USA, February 13, 2016*,
1174 volume WS-16-12 of *AAAI Technical Report*. AAAI Press, 2016. URL: [http://www.aaai.org/](http://www.aaai.org/ocs/index.php/WS/AAAIW16/paper/view/12611)
1175 [ocs/index.php/WS/AAAIW16/paper/view/12611](http://www.aaai.org/ocs/index.php/WS/AAAIW16/paper/view/12611).
- 1176 **34** Christos H. Papadimitriou, Prabhakar Raghavan, Madhu Sudan, and Hisao Tamaki. Motion
1177 planning on a graph (extended abstract). In *STOC*, pages 511–520, 1994.
- 1178 **35** Daniel Ratner and Manfred Warmuth. The $(n^2 - 1)$ -puzzle and related relocation problems.
1179 *Journal of Symbolic Computation*, 10(2):111–137, 1990.
- 1180 **36** Roni Stern, Nathan R. Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne T. Walker,
1181 Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Roman Barták, and Eli Boyarski.
1182 Multi-agent pathfinding: Definitions, variants, and benchmarks. In *Proceedings of the Twelfth*
1183 *International Symposium on Combinatorial Search (SOCS)*, pages 151–158. AAAI Press, 2019.
1184 URL: <https://doi.org/10.1609/socs.v10i1.18510>, doi:10.1609/SOCS.V10I1.18510.
- 1185 **37** Pavel Surynek. An optimization variant of multi-robot path planning is intractable. In
1186 *Proceedings of the AAAI conference on artificial intelligence*, volume 24, pages 1261–1263,
1187 2010.
- 1188 **38** Sumanth Varambally, Jiaoyang Li, and Sven Koenig. Which MAPF model works best for
1189 automated warehousing? In Lukás Chrpá and Alessandro Saetti, editors, *Proceedings of the*
1190 *Fifteenth International Symposium on Combinatorial Search, SOCS 2022, Vienna, Austria,*
1191 *July 21-23, 2022*, pages 190–198. AAAI Press, 2022. URL: [https://doi.org/10.1609/socs.](https://doi.org/10.1609/socs.v15i1.21767)
1192 [v15i1.21767](https://doi.org/10.1609/socs.v15i1.21767), doi:10.1609/SOCS.V15I1.21767.
- 1193 **39** Manuela M. Veloso, Joydeep Biswas, Brian Coltin, and Stephanie Rosenthal. Cobots: Robust
1194 symbiotic autonomous mobile service robots. In Qiang Yang and Michael J. Wooldridge, editors,
1195 *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence,*
1196 *IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, page 4423. AAAI Press, 2015. URL:
1197 <http://ijcai.org/Abstract/15/656>.
- 1198 **40** Peter Wurman, Raffaello D’Andrea, and Mick Mountz. Coordinating hundreds of cooperative,
1199 autonomous vehicles in warehouses. *AI Magazine*, 29:9–20, 03 2008.
- 1200 **41** Jingjin Yu and Steven M. LaValle. Structure and intractability of optimal multi-robot path
1201 planning on graphs. In Marie desJardins and Michael L. Littman, editors, *AAAI*. AAAI Press,
1202 2013.